ADITI: Adaptive Drift-aware Intervals for Time-series Inference

Ayush Upadhyay
School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, India
ayushdevxai@gmail.com

Abstract. *Prediction intervals for monitoring metrics must hold their coverage when it matters most: during incidents, when residual scale jumps and the stream stops resembling its past. Online conformal methods such as adaptive conformal inference guarantee long-run coverage for any sequence, but they adapt only after observing misses and can under-cover for hundreds of steps after a change. We present ADITI (Adaptive Drift-aware Intervals for Time-series Inference), an online conformal method that calibrates residuals normalized by a running robust scale, uses an exponentially weighted conformal quantile whose memory shortens when a two-sided CUSUM detector flags a change in the score distribution, and keeps an ACI-style coverage feedback. We prove a long-run coverage bound that degrades with the number of drift alarms, extend it to a width cap, and identify a multiplicative width blow-up that normalized scores can suffer. On the 47 real series of the Numenta Anomaly Benchmark, with every hyperparameter chosen on 15 development series and a single run on 32 held-out test series, ADITI with an AR forecaster reached the lowest local coverage error (0.0121 against 0.0144 to 0.0280 for rolling split conformal, ACI, DtACI and quantile tracking; better than each on 25 to 31 series), was the only method at the 90% target inside labelled anomaly windows (0.901 against 0.848 to 0.885), and had the lowest median interval score. It also failed: on two heavy-tailed series its intervals blew up, which a width cap added after the test run repairs on one and bounds on the other; with a naive forecaster quantile tracking was steadier; and at an 80% target the advantage vanished. An ablation and controlled experiments with known changes show that normalization and feedback carry most of the gain; the drift detector rarely fires once residuals are normalized and helps only with the weaker forecaster.*

Index Terms. *Conformal prediction, prediction intervals, time series, distribution shift, change detection, CUSUM, anomaly detection, operational monitoring.*

Provenance note. All results come from real NAB data (repository commit ea702d7) and a single run on held-out test series with settings frozen on development series. One component, the width cap of Section IV-E, was added after the test run; results are reported with and without it. Code regenerates every number, table and figure.

I. Introduction

Monitoring systems watch thousands of sensor and service metrics: CPU utilization, request latency, machine temperature, traffic speed, message volume. For each metric, an operator wants more than a point forecast: a band that the next observation should fall in, and an alert when it does not. The band is only useful if its coverage is honest. A band that claims 90% coverage but covers 60% during an incident produces a flood of false alerts exactly when attention is scarcest, and a band that is wide enough never to miss hides the incident itself.

Conformal prediction gives distribution-free coverage guarantees [1, 2], but its classical form assumes exchangeable data. Operational streams are not exchangeable: they drift, switch regimes and spike. Online conformal methods restore a long-run guarantee by adjusting the target level with feedback. Adaptive conformal inference (ACI) raises the nominal level after misses and lowers it after hits [5]; dynamically tuned ACI (DtACI) aggregates several step sizes [6]; quantile tracking and conformal PID control update the threshold directly [7]; strongly adaptive methods bound regret on every interval [8]. These guarantees are *long-run averages*: over thousands of steps the miss rate converges to α. They do not prevent long stretches of under-coverage after a change, which is precisely when an operator needs the band most.

This paper presents **ADITI** (**A**daptive **D**rift-aware **I**ntervals for **T**ime-series **I**nference), an online conformal method built for that failure mode. ADITI combines three ingredients, each simple and individually known, in a way we found works well on real operational data:

1. *scale normalization*: residuals are divided by a running robust scale estimate, so the band widens and narrows with local volatility instead of waiting for misses;
2. *drift-aware memory*: the calibration quantile is an exponentially weighted quantile whose forgetting rate switches from slow to fast when a two-sided CUSUM detector [9] flags a change in the score distribution, then relaxes back;
3. *coverage feedback*: an ACI-style update of the target level [5], which restores a long-run coverage guarantee.

A. Research questions

RQ1 On real operational streams, does ADITI hold coverage more steadily over time than rolling, ACI, DtACI and quantile-tracking intervals, without wider bands?

RQ2 Does it hold coverage during and after labelled anomalies, when intervals matter most?

RQ3 Which of its three ingredients are responsible?

RQ4 How robust are the results to the forecaster, the coverage level, the data category and the hyperparameters, and where does ADITI fail?

B. Contributions

1. **ADITI**, an online conformal interval method with scale normalization, CUSUM-triggered adaptive memory and coverage feedback (Section IV).
2. **Guarantees**: a long-run coverage bound that degrades gracefully with the number of drift alarms, its extension to

the width cap, the effective calibration sample size of each memory regime, and an analysis of a multiplicative width blow-up that normalized scores can suffer on heavy-tailed streams (Section V).

3. **A held-out evaluation** on the 47 real series of the Numenta Anomaly Benchmark (NAB) [10]: all hyperparameters chosen on 15 development series, then a single run on 32 untouched test series with paired tests across series (Sections VI–VII).
4. **Honest findings, positive and negative**: ADITI gives the lowest local coverage error and the highest coverage inside anomaly windows; it has a catastrophic width failure on 2 of 32 series that a one-line cap repairs, which we report as a post-hoc fix; its advantage shrinks or reverses with a naive forecaster and at a low coverage target; and controlled experiments show that its drift detector rarely fires once residuals are normalized (Sections VIII and IX).

II. Background and Related Work

A. Conformal prediction

Given a score $s(x,y)$ and n exchangeable calibration scores, the set $\{y : s(x,y) \le \hat{Q}\}$ with $\hat{Q}$ the $\lceil (n+1)(1-\alpha) \rceil$-th smallest score has coverage at least $1-\alpha$ [1, 2]; tutorial treatments are given in [3, 4]. Normalized scores $s/\hat{\sigma}(x)$ make the interval width adapt to local difficulty [2, 11], and conformalized quantile regression achieves a similar effect with quantile models [12]. Weighted conformal prediction handles known covariate shift [13], and Barber et al. bound the coverage loss of weighted quantiles under arbitrary non-exchangeability by a weighted sum of total-variation distances [14]; exponentially decaying weights, as in ADITI, are one of their motivating examples.

B. Online conformal inference under shift

ACI [5] updates the target level $\alpha_{t+1} = \alpha_t + \gamma(\alpha - \mathrm{err}_t)$ and guarantees $|\frac{1}{T}\sum_t \mathrm{err}_t - \alpha| \le \frac{\max(\alpha_1, 1-\alpha_1)+\gamma}{\gamma T}$ for any sequence. Its step size γ trades adaptivity against stability; DtACI [6] runs several step sizes and combines them with exponential weights, and AgACI [15] aggregates experts online. Strongly adaptive online conformal prediction (SAOCP) bounds regret on every interval [8]. Conformal PID control [7] tracks the quantile directly, adds integral and "scorecasting" terms, and is the basis of our quantile-tracking baseline. EnbPI builds bootstrap ensembles for time series [16], SPCI fits a quantile model to the sequence of residuals [17], and Stankevičiūtė et al. apply conformal prediction to multi-horizon forecasting [18]. These methods adapt through the miss indicator, a fixed calibration window or a learned residual model; none of them detects a change in the score distribution and changes its memory in response.

C. Change detection

CUSUM [9] accumulates standardized deviations and alarms when the sum crosses a threshold; it is optimal for detecting a known mean shift and robust in practice [19]. Concept-drift adaptation in learning systems often pairs a detector with window resizing [20]. ADITI applies the same pattern to the calibration memory of a conformal procedure.

Table I
ONLINE INTERVAL METHODS COMPARED

Method	Long-run guarantee	Local adaptivity	Drift detection	Scale-adaptive	Tuning
Rolling split	–	window	–	–	W
ACI [5]	✓	via γ	–	–	γ
DtACI [6]	✓	multi-γ	–	–	grid
SAOCP [8]	✓	all intervals	–	–	lifetime
Conformal PID [7]	✓	via step	–	range-scaled	η
ADITI	✓	memory + γ	✓	✓	$\lambda_s, \lambda_f, h, \gamma, \beta$

Table II
NOTATION

Symbol	Meaning
y_t, $\hat{y}_t$	observation and one-step forecast
$s_t = \lvert y_t - \hat{y}_t \rvert$	nonconformity score
$\hat{\sigma}_t$	running robust scale; $r_t = s_t/\hat{\sigma}_t$ normalized score
α, α_t	target and adjusted miscoverage level
γ	feedback step size
λ_s, λ_f	slow and fast forgetting factors
h, k	CUSUM threshold and allowance
R	fast-memory duration after an alarm (steps)
β	scale-update rate; W memory length
$\hat{q}_t$	normalized quantile; half-width $\hat{\sigma}_t \hat{q}_t$
err_t	miss indicator $\mathbf{1}[s_t > \text{half-width}_t]$
N_T	number of drift alarms up to T

D. Anomaly benchmarks

NAB provides real streaming series with labelled anomaly windows [10, 21]. We use NAB not to score anomaly detectors but because its labels mark exactly the moments when interval coverage is hardest and most valuable.

Table I positions ADITI.

III. Problem Formulation

Table II lists the notation.

A. Online protocol

At each step t the method sees the past $y_{<t}$, the forecast $\hat{y}_t$, and its own history of scores; it issues a symmetric interval $[\hat{y}_t - u_t, \hat{y}_t + u_t]$; then y_t is revealed and $\mathrm{err}_t = \mathbf{1}[s_t > u_t]$. No method sees y_t before issuing u_t. The first 300 steps of each series are a warm-up and are not scored.

B. Metrics

Let $\mathscr{T}$ be the scored steps of a series. **Marginal coverage** is $\bar{c} = \frac{1}{|\mathscr{T}|}\sum_{t\in\mathscr{T}}(1-\mathrm{err}_t)$. **Local coverage** over a window of $m = 250$ steps is $c_t^{(m)} = \frac{1}{m}\sum_{i=t-m+1}^{t}(1-\mathrm{err}_i)$, and we report

$$\text{local coverage error} = \frac{1}{|\mathscr{T}_m|}\sum_t \left| c_t^{(m)} - (1-\alpha) \right|, \quad (1)$$

$$\text{under-coverage share} = \frac{1}{|\mathscr{T}_m|}\sum_t \mathbf{1}\left[c_t^{(m)} < 0.8 \right], \quad (2)$$

plus the worst local coverage $\min_t c_t^{(m)}$. **Efficiency** uses the interval (Winkler) score [23, 24],

$$S_t = 2u_t + \tfrac{2}{\alpha}\max(0, s_t - u_t), \tag{3}$$

a proper scoring rule that rewards narrow intervals and penalizes misses in proportion to their size. Because series have incomparable scales, we compare methods through the ratio of each method's mean interval score to that of the rolling baseline on the same series, aggregated across series by geometric mean and median. **Anomaly-window coverage** is coverage inside NAB's labelled windows, and **recovery** is the number of steps after a window ends until 50-step coverage returns to 0.85.

C. Goal

We seek intervals whose local coverage stays close to $1-\alpha$ at all times, especially inside anomaly windows, at an interval score no worse than the baselines, while keeping a long-run guarantee.

IV. ADITI

Fig. 1 shows the components and Fig. 4 one step.

A. Scale normalization

ADITI keeps a robust running scale

$$\hat{\sigma}_{t+1} = (1-\beta)\hat{\sigma}_t + \beta\,\min(s_t,\ \kappa\hat{\sigma}_t), \tag{4}$$

an exponentially weighted mean of absolute residuals, optionally winsorized at $\kappa\hat{\sigma}_t$ (we use $\kappa = 4$ for the naive forecaster and no clipping for the AR forecaster, as selected on development data). It calibrates on normalized scores $r_t = s_t/\hat{\sigma}_t$ and issues half-width $u_t = \hat{\sigma}_t\hat{q}_t$. When volatility rises, $\hat{\sigma}_t$ rises with it and the band widens immediately, before any miss is observed.

B. Drift-aware weighted quantile

With stored normalized scores $r_{t-W}, \ldots, r_{t-1}$ and forgetting factor λ_t, ADITI assigns weights $w_i = \lambda_t^{\,t-1-i}$ and computes the weighted conformal quantile

$$\hat{q}_t = \inf\Big\{q : \sum_i \tilde{w}_i \mathbf{1}[r_i \le q] \ge 1-\alpha_t\Big\}, \quad \tilde{w}_i = \frac{w_i}{1+\sum_j w_j}, \tag{5}$$

where the extra unit of mass stands for the test point [13, 14]; $\hat{q}_t = \infty$ if the level cannot be reached and 0 if $\alpha_t \ge 1$. The forgetting factor is

$$\lambda_t = \begin{cases} \lambda_f, & \text{an alarm occurred in the last } R \text{ steps,} \\ \lambda_s, & \text{otherwise,} \end{cases} \tag{6}$$

with $\lambda_s = 0.995$ (half-life 138 steps), $\lambda_f = 0.98$ (half-life 34) and $R = 200$ (Fig. 2 and Fig. 3).

C. Drift detector

ADITI standardizes the log normalized score with slow running moments,

$$z_t = \frac{\log(r_t + \varepsilon) - \mu_t}{\varsigma_t}, \tag{7}$$

Algorithm 1 ADITI, one time step

Require: forecast $\hat{y}_t$; memory $\{r_i\}$; $\hat{\sigma}_t$, α_t, $C^{\pm}$, timer ρ
1: $\lambda_t \leftarrow \lambda_f$ if $\rho < R$ else λ_s
2: $\hat{q}_t \leftarrow$ weighted quantile (5) at level $1-\alpha_t$
3: $u_t \leftarrow \min(\hat{\sigma}_t\hat{q}_t,\ \max_i s_i)$ ▷ cap (10)
4: **issue** $[\hat{y}_t - u_t,\ \hat{y}_t + u_t]$
5: observe y_t; $s_t \leftarrow |y_t - \hat{y}_t|$; $r_t \leftarrow s_t/\hat{\sigma}_t$; $\mathrm{err}_t \leftarrow \mathbf{1}[s_t > u_t]$
6: $\alpha_{t+1} \leftarrow \alpha_t + \gamma(\alpha - \mathrm{err}_t)$
7: update $C^{\pm}$ by (8); $\rho \leftarrow \rho + 1$
8: **if** $C^+ > h$ or $C^- > h$ **then**
9: $\quad C^{\pm} \leftarrow 0$; $\rho \leftarrow 0$; $\alpha_{t+1} \leftarrow \alpha$
10: **end if**
11: append r_t; update μ, ς (rate 0.005) and $\hat{\sigma}_{t+1}$ by (4)

and runs a two-sided CUSUM [9]

$$C_t^+ = \max(0, C_{t-1}^+ + z_t - k), \qquad C_t^- = \max(0, C_{t-1}^- - z_t - k), \tag{8}$$

with allowance $k = 0.5$. An alarm fires when $C_t^+ > h$ or $C_t^- > h$; it resets both sums, starts the fast-memory period and resets the feedback level $\alpha_t \leftarrow \alpha$. The log transform makes multiplicative changes in residual size additive, and the two sides catch both rising and falling volatility.

D. Coverage feedback

After observing y_t,

$$\alpha_{t+1} = \alpha_t + \gamma(\alpha - \mathrm{err}_t), \tag{9}$$

exactly as in ACI [5]. Normalization and memory make $\hat{q}_t$ track the right shape; feedback corrects any remaining bias in its level.

E. Width cap (post-hoc safeguard)

Our test run exposed a failure on 2 of 32 series (Section VII-E), which a cap repairs:

$$u_t \leftarrow \min\Big(u_t,\ \max_{t-W \le i < t} s_i\Big). \tag{10}$$

The cap never makes an interval wider than the largest raw residual in memory, which is what an unnormalized conformal interval at full level would give. We added it after seeing test results; it is reported separately and checked on development data first.

F. Algorithm

G. Complexity

The weighted quantile sorts at most $W = 1000$ stored scores, $O(W \log W)$ per step; everything else is $O(1)$. A sliding order-statistics tree would make the quantile $O(\log W)$; our Python reference takes 67 μs per step (Section VII-D.7).

V. Analysis

Lemma 1 (Monotone quantile map)**.** *For fixed memory and weights, $\hat{q}_t$ in (5) is non-increasing in α_t, equals ∞ for $\alpha_t < \frac{1}{1+\sum_j w_j}$ and equals 0 for $\alpha_t \ge 1$.*

Proof. The constraint $\sum_i \tilde{w}_i \mathbf{1}[r_i \le q] \ge 1-\alpha_t$ becomes weaker as α_t grows, so the infimum cannot increase. The left side is at

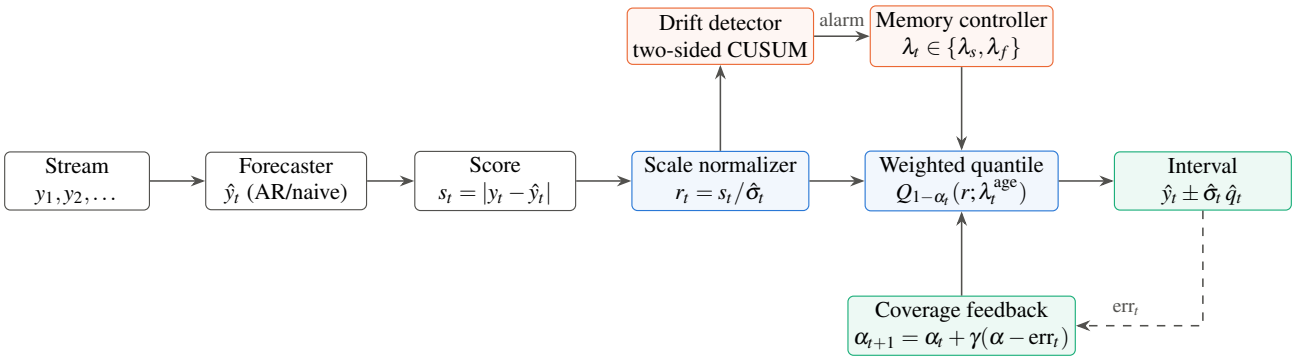


Figure 1. ADITI architecture. Scores are normalized by a running scale; a CUSUM detector on the normalized scores switches the calibration memory between slow and fast forgetting (and resets the feedback level on each alarm); an ACI-style feedback loop on the miss indicator keeps long-run coverage.

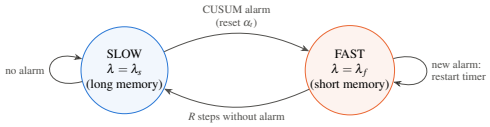


Figure 2. Memory controller as a two-state machine.

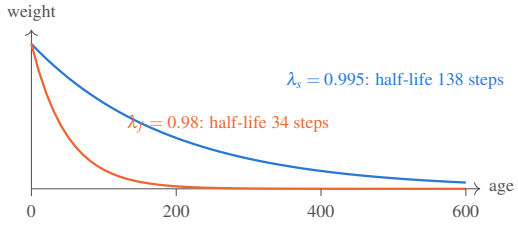


Figure 3. Calibration weights by score age in the slow and fast regimes.

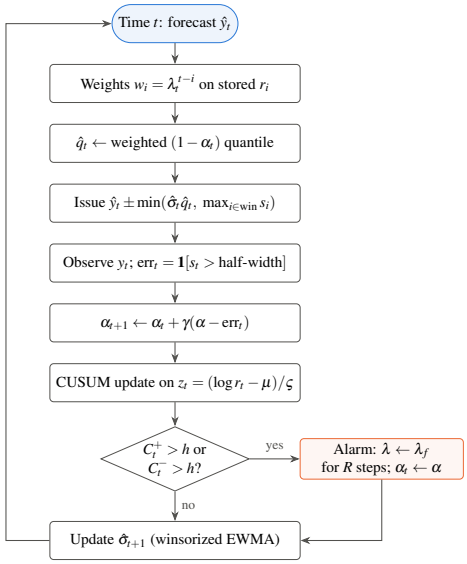


Figure 4. Control flow of one ADITI step.

most $\sum_i w_i/(1+\sum_j w_j)$, so for $1-\alpha_t$ above that value no finite q satisfies it; for $\alpha_t \geq 1$ the constraint holds at $q = 0$. □

Theorem 1 (Long-run coverage with resets)**.** *Without the cap, for any sequence of scores, ADITI's miss indicators satisfy*

$$\left| \frac{1}{T} \sum_{t=1}^{T} \text{err}_t - \alpha \right| \leq \frac{(N_T + 1)(1 + 2\gamma)}{\gamma T}, \tag{11}$$

where N_T is the number of drift alarms in the first T steps.

Proof. First, $\alpha_t \in [-\gamma, 1+\gamma]$ for all t. If $\alpha_t < 0$ then by Lemma 1 $\hat{q}_t = \infty$, so $\text{err}_t = 0$ and $\alpha_{t+1} = \alpha_t + \gamma\alpha > \alpha_t$; hence α_t can fall below 0 by at most one step of size $\gamma(1-\alpha) < \gamma$. Symmetrically, if $\alpha_t \geq 1$ then $u_t = 0$, so $\text{err}_t = 1$ whenever $s_t > 0$ and $\alpha_{t+1} < \alpha_t$; a zero score gives $\text{err}_t = 0$ and moves α_t up by at most $\gamma\alpha$, which the same argument bounds. Resets set $\alpha_t = \alpha \in [0,1]$. Second, split $[1,T]$ at the alarm times into $N_T + 1$ segments $[a,b]$. On each segment no reset occurs, so summing (9) gives $\sum_{t=a}^{b}(\alpha - \text{err}_t) = (\alpha_{b+1} - \alpha_a)/\gamma$, with $|\alpha_{b+1} - \alpha_a| \leq 1 + 2\gamma$. Summing over segments and dividing by T gives the bound. □

With $N_T = 0$ this recovers the ACI rate; each alarm costs one extra ACI "budget". On our test series ADITI raised 156 alarms over 228,748 steps (0.68 per 1000 steps), so with $\gamma = 0.005$ the bound on a 7,000-step series with 5 alarms is $6 \cdot 1.01/(0.005 \cdot 7000) = 0.17$, loose but vanishing as T/N_T grows.

Corollary 1 (With the cap)**.** *With the cap* (10)*, let K_T be the number of steps at which s_t exceeds every score in memory (a new record). Then*

$$\left| \frac{1}{T} \sum_{t} \text{err}_t - \alpha \right| \leq \frac{(N_T + 1)(1 + 2\gamma)}{\gamma T} + \frac{(N_T + 1)K_T}{T}. \tag{12}$$

Proof. The upper bound $\alpha_t \leq 1 + \gamma$ is unaffected. For the lower bound, if $\alpha_t < 0$ the capped half-width equals the largest stored score, so a miss is possible only if s_t is a new record; at every other step with $\alpha_t < 0$ there is no miss and α_t rises. Each record step can lower α_t by at most γ, so $\alpha_t \geq -\gamma(1 + K_T)$.

On each of the $N_T + 1$ reset-free segments, $|\alpha_{b+1} - \alpha_a| \leq 1 + 2\gamma + \gamma K_T$, and the segment-sum argument of Theorem 1 gives the bound. □

For i.i.d. continuous scores, a new maximum of the last W scores occurs with probability $1/(W+1)$ per step, so $K_T/T \approx 1/1001$ with $W = 1000$: the cap's cost to the guarantee is small unless alarms are frequent.

Proposition 1 (Effective calibration size)**.** *With weights λ^a over ages $a = 0, 1, \ldots$, the effective sample size $n_{eff} = (\sum w)^2 / \sum w^2$ tends to $(1+\lambda)/(1-\lambda)$ as $W \to \infty$: 399 scores for $\lambda_s = 0.995$ and 99 for $\lambda_f = 0.98$.*

Proof. $\sum_a \lambda^a = 1/(1-\lambda)$ and $\sum_a \lambda^{2a} = 1/(1-\lambda^2)$, so $n_{\text{eff}} = (1-\lambda^2)/(1-\lambda)^2 = (1+\lambda)/(1-\lambda)$. □

The fast regime thus calibrates on about 100 recent scores, enough to estimate a 90% quantile with roughly 10 exceedances, while discarding pre-change scores four times faster than the slow regime. For $W = 1000$ and $\lambda_s = 0.995$ the truncation leaves $1 - 0.995^{1000} = 99.3\%$ of the infinite-history weight, so the limit is accurate.

Proposition 2 (Local adaptivity of normalization)**.** *Suppose $s_t = \sigma_t \varepsilon_t$ with ε_t i.i.d. and $\hat{\sigma}_t = c\,\sigma_t$ for a constant $c > 0$. Then the normalized scores $r_t = \varepsilon_t / c$ are i.i.d., and the unweighted split-conformal interval on r (with $\alpha_t = \alpha$) has coverage at least $1 - \alpha$ conditionally on the scale path $\{\sigma_t\}$.*

Proof. Conditioning on $\{\sigma_t\}$ leaves $\{r_t\}$ i.i.d., hence exchangeable, and the standard split-conformal argument applies to r. The event $r_t \leq \hat{q}$ equals $s_t \leq \hat{\sigma}_t \hat{q}$. □

Unnormalized methods meet the marginal target only on average over volatility regimes: they over-cover in calm periods and under-cover in volatile ones. Normalization removes this dependence to the extent that $\hat{\sigma}_t$ tracks σ_t, which is why it carries most of ADITI's local-coverage gain in the ablation (Section VII-C).

Proposition 3 (Multiplicative width blow-up)**.** *Let a score s_i arrive when the scale estimate is $\hat{\sigma}_i$, and suppose it stays in memory and determines $\hat{q}_t = r_i = s_i/\hat{\sigma}_i$ at a later time with scale $\hat{\sigma}_t$. Then the half-width is $u_t = s_i \hat{\sigma}_t / \hat{\sigma}_i$, which exceeds every raw score in memory by the factor $\hat{\sigma}_t/\hat{\sigma}_i$ if s_i was the largest.*

Proof. Direct substitution: $u_t = \hat{\sigma}_t \hat{q}_t = \hat{\sigma}_t s_i / \hat{\sigma}_i$. □

On a stream that is flat for a long time ($\hat{\sigma}_i$ tiny), then spikes, then turns volatile ($\hat{\sigma}_t$ large), the factor can be enormous. That is exactly what happened on one test series (Section VII-E), and the cap (10) bounds u_t by the largest raw score, removing the factor.

VI. Experimental Setup

Fig. 5 summarizes the protocol.

Table III
NAB real-data series used

Category	Dev	Test	Dev points	Test points	Windows
AWS CloudWatch	6	11	24,890	42,850	30
Ad exchange	2	4	3,267	6,343	14
Known cause	2	5	12,202	57,359	19
Traffic	2	5	4,542	11,122	14
Tweets	3	7	47,557	111,074	33
Total	15	32	92,458	228,748	110

Windows: labelled anomaly windows across dev and test series.

A. Data

We use all real-data series of NAB [10] (repository commit ea702d7): 47 series, 321,206 observations, in five categories (Table III). Artificial series are excluded. NAB's labelled anomaly windows, drawn around known incidents, serve as ground truth for where coverage is hardest. We split series 1:2 into development and test sets, stratified by category with a fixed seed, giving 15 development and 32 test series.

B. Forecasters

Interval methods wrap a point forecaster; we use two standard choices [22]. **AR**: an online AR(12) model with intercept, fitted by recursive least squares with forgetting factor 0.999 on a running-standardized series, with a reset on numerical blow-up. **Naive**: $\hat{y}_t = y_{t-1}$. Neither is tuned to the data; the AR forecaster has lower mean absolute error than the naive one on 20 of 32 test series (geometric-mean MAE ratio 0.73 on known-cause and 0.89 on traffic series, but 1.44 on AWS series, where the naive forecaster is better). We treat AR as the primary forecaster and report naive results as a robustness check.

C. Baselines

Rolling split: the split-conformal quantile of the last W scores. **ACI** [5]: rolling quantile at level $1 - \alpha_t$ with the feedback (9). **DtACI** [6]: eight ACI experts with $\gamma \in \{0.001, 0.002, \ldots, 0.128\}$ combined by exponential weights with the learning rate and mixing parameter recommended by the authors. **Quantile tracking** [7]: the P-control update $q_{t+1} = q_t + \eta B_t(\text{err}_t - \alpha)$ with B_t the largest recent score. Infinite intervals (ACI, DtACI when $\alpha_t < 0$) are counted at the largest finite width of that run when computing widths and interval scores, and their frequency is reported.

D. Hyperparameter selection

For each method and forecaster we ran a grid on the development series and kept the setting with the lowest geometric-mean interval score relative to Rolling ($W = 500$), among settings whose mean coverage was within 0.02 of the target. Table IV lists the selected values. ADITI's grid was run in two rounds on development data: the second round added the winsorized scale update after the first showed weak results with the naive forecaster. No test series was loaded during selection.

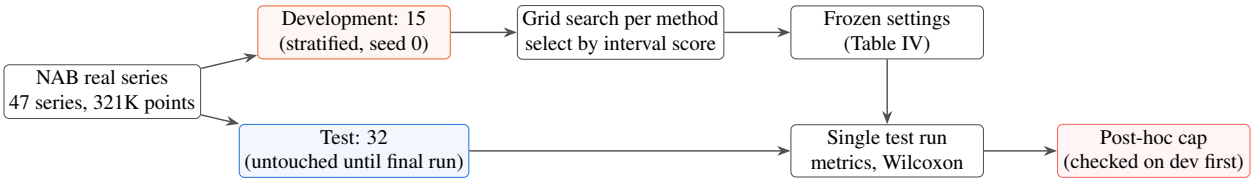


Figure 5. Evaluation protocol. Every hyperparameter is chosen on development series; test series are used once. The width cap was added after the test run and is reported separately.

Table IV
SELECTED HYPERPARAMETERS (DEVELOPMENT SERIES)

Method	AR forecaster	Naive forecaster
Rolling split	$W = 250$	$W = 250$
ACI	$\gamma = 0.01$, $W = 500$	$\gamma = 0.01$, $W = 1000$
DtACI	$W = 500$	$W = 1000$
Quantile tracking	$\eta = 0.01$	$\eta = 0.03$
ADITI	$\lambda_s = 0.995$, $\lambda_f = 0.98$, $h = 12$, $\gamma = 0.005$, $\beta = 0.1$, no clipping, $W = 1000$, $R = 200$	same, but $\beta = 0.02$, $\gamma = 0.01$, $\kappa = 4$

Grids: Rolling $W \in \{250, 500, 1000\}$; ACI $\gamma \in \{0.002, .., 0.02\}$; QT $\eta \in \{0.01, .., 0.3\}$; ADITI $h \in \{5, 12\}$, $\beta \in \{0.02, 0.1\}$, $\gamma \in \{0.005, 0.01\}$, $\kappa \in \{\text{none}, 4\}$ (round 2).

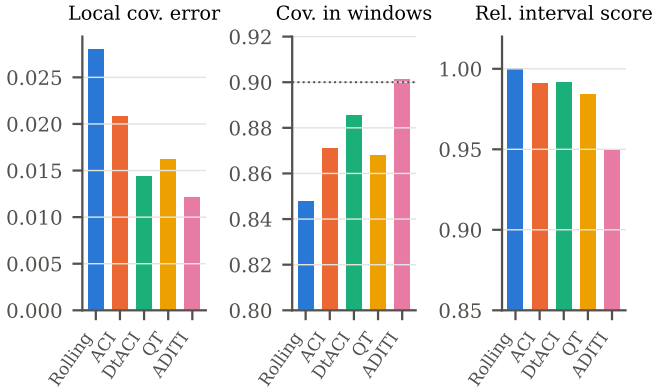


Figure 6. Local coverage error, coverage inside anomaly windows (dotted line: target) and median relative interval score (ADITI with cap), AR forecaster, 32 test series.

E. Statistics

Each test series is one paired observation (32 pairs). We use two-sided Wilcoxon signed-rank tests [25] on per-series metrics (log ratios for interval scores) and report the number of series on which ADITI is better. The target coverage is $1 - \alpha = 0.9$ unless stated; Section VII-D repeats the evaluation at 0.95 and 0.8. Runtimes are medians per step in single-threaded Python on a 2-vCPU container.

VII. RESULTS

A. RQ1: Coverage stability and efficiency

Table V gives the test results with the AR forecaster, and Fig. 6 the three headline metrics. All five methods reach the 90% target on average (0.897–0.901). They differ in how steadily they hold it. ADITI has the lowest local coverage error (0.0121, against 0.0144 for DtACI, 0.0162 for quantile tracking, 0.0209 for ACI and 0.0280 for rolling) and the highest worst-case local coverage (0.859 on average across series). DtACI has the fewest 250-step windows below 80% coverage (none), ADITI and quantile tracking are next (0.22% and 0.16%). ADITI's bands are also narrower than every baseline except DtACI: geometric-mean width 0.867 of the rolling baseline, against 0.858 for DtACI, 0.944 for ACI and 1.016 for quantile tracking.

Interval score and the two failures: Efficiency splits into two stories. On the typical series ADITI is the most efficient method: its median interval score is 5.1% below the rolling baseline, against 0.8–1.6% for the other methods, and it beats every baseline on a majority of series (Table VI). But the as-selected ADITI's geometric mean is 2.06, because on two series its intervals blew up (Section VII-E). With the post-hoc width cap the geometric mean is 0.995, level with ACI (0.994) and slightly behind quantile tracking (0.978), and the median is unchanged. DtACI's geometric mean of 1.190 has a different cause: it issues infinite intervals 1.05% of the time, which the interval score charges at the largest finite width.

Fig. 8 shows the distribution of local coverage error across series: ADITI's median and upper quartile are the lowest, and its worst series (0.040) matches DtACI's (0.040) and is well below rolling (0.088) and ACI (0.066), though quantile tracking has the mildest worst case (0.026). Fig. 9 plots each series' efficiency against its local error; ADITI's points concentrate in the lower left, with the exception of one series to the right (the remaining outlier after the cap, a 6.3× ratio on `ec2_disk_write_bytes_1ef3de`).

Fig. 7 shows the same steadiness in time: for each test series and each of 120 time bins it plots the lowest 250-step coverage in the bin. Rolling intervals show long red stretches of under-coverage and blue stretches of over-coverage on the same series; quantile tracking narrows both; ADITI's rows are close to uniform, with visible excursions on a few series only.

B. RQ2: Coverage around anomalies

NAB's labelled windows mark incidents: failures, misconfigurations, outages, traffic events. Inside these windows ADITI's coverage averages 0.901, the only method at the target, against 0.885 for DtACI, 0.871 for ACI, 0.868 for quantile tracking and 0.848 for rolling; it is better on 21–29 of 31 series against each baseline, all with $p \leq 0.001$ (Table VI).

Fig. 10 aligns all 55 test windows with at least 200 steps of history and 600 steps of follow-up at their start, and averages coverage and width; Table VII summarizes it. Before the

Table V

TEST RESULTS, AR FORECASTER, TARGET COVERAGE 0.9 (32 SERIES; MEANS OVER SERIES UNLESS STATED)

Method	Coverage	Local cov. error ↓	Windows <80% ↓	Worst local ↑	Interval score / Rolling geo. mean	Interval score / Rolling median	Width / Rolling	Cov. in windows ↑	Inf.	μs/step
Rolling split	0.897	0.0280	1.65%	0.786	1.000	1.000	1.000	0.848	0.00%	2.4
ACI	0.900	0.0209	0.74%	0.824	0.994	0.991	0.944	0.871	0.03%	3.4
DtACI	0.901	0.0144	**0.00%**	0.852	1.190	0.992	**0.858**	0.885	1.05%	18.2
Quantile tracking	0.899	0.0162	0.16%	0.831	**0.978**	0.984	1.016	0.868	0.00%	2.1
ADITI (as selected)	0.899	**0.0121**	0.22%	**0.859**	2.061	**0.949**	0.867	**0.901**	0.00%	66.8
ADITI + cap (post-hoc)	0.899	**0.0121**	0.22%	**0.859**	0.995	**0.949**	0.867	**0.901**	0.00%	–

Interval scores relative to Rolling split on the same series. Inf.: share of infinite intervals. "Windows <80%": share of 250-step windows with coverage below 0.8.

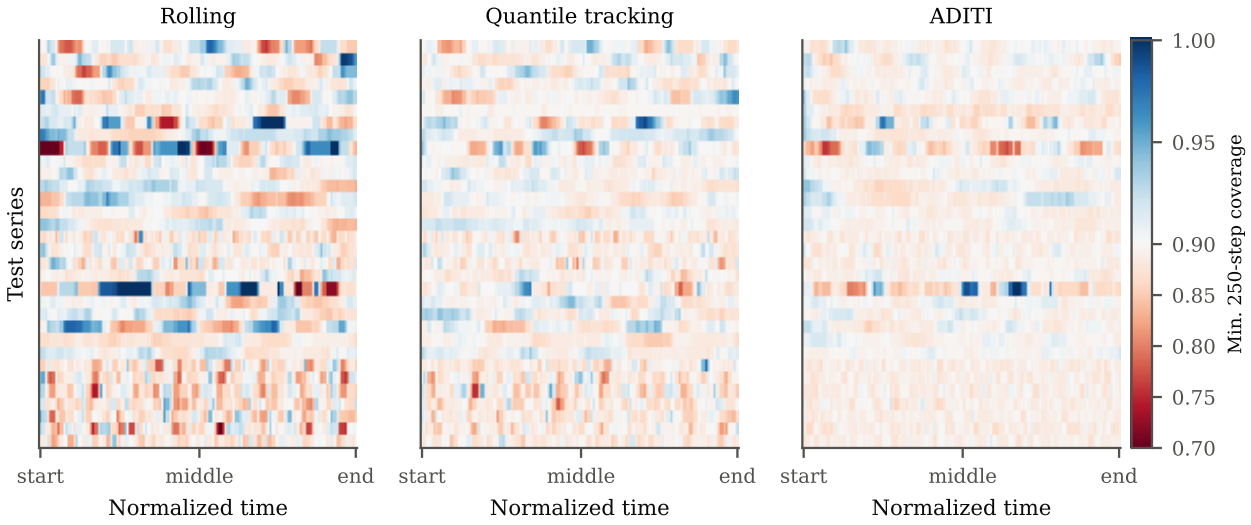


Figure 7. Lowest 250-step coverage in each of 120 time bins, one row per test series (AR forecaster, same row order in each panel). White is the 0.9 target, red under-coverage, blue over-coverage.

Table VI

PAIRED COMPARISON ON 32 TEST SERIES, ADITI VS. EACH BASELINE (AR)

Metric	Rolling	ACI	DtACI	QT
Interval score	24/32 (.002)	25/32 (.002)	21/32 (.039)	24/32 (.004)
Local cov. error	31/32 (<.001)	30/32 (<.001)	25/32 (.003)	27/32 (.003)
Windows <80%	12/32 (.002)	5/32 (.17)	0/32 (.18)	4/32 (.50)
Cov. in windows	29/31 (<.001)	21/31 (<.001)	22/31 (<.001)	29/31 (<.001)

Entries: series where ADITI is better / series compared (p, two-sided Wilcoxon).
Ties count as not better; "windows <80%" is often 0 for both methods.

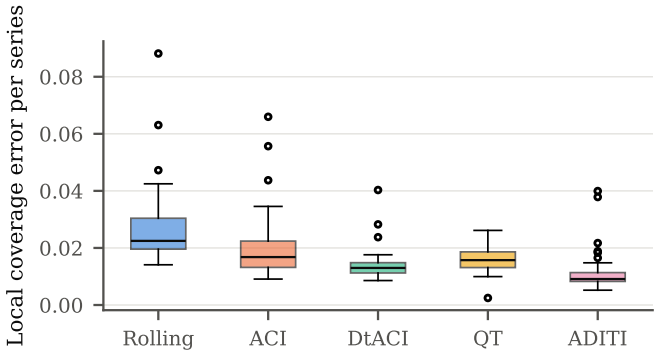


Figure 8. Per-series local coverage error (AR forecaster, 32 test series).

window all methods sit near 0.90. In the first 100 steps of the window, rolling coverage falls to 0.870 and keeps falling to 0.859 over the next 200 steps; ACI and quantile tracking dip to about 0.88. ADITI stays at 0.897 and 0.903. The width panel shows a second difference: all methods widen 150–200 steps into the window, but ADITI's width falls back within about 100 steps, while rolling and quantile-tracking widths stay high until 400–500 steps, after the coverage dip has passed.

After windows end, the ordering reverses slightly: in the 500 steps after a window ADITI covers 0.895, the baselines 0.905–0.909 (Table V does not show this column; see Appendix A). ADITI's fast memory has by then adapted to the

Table VII

COVERAGE ALIGNED AT ANOMALY-WINDOW START (55 WINDOWS)

Method	$[-200, 0)$	$[0, 100)$	$[100, 300)$	$[300, 600)$
Rolling split	0.907	0.870	0.859	0.912
ACI	0.906	0.881	0.886	0.916
DtACI	0.901	0.895	0.896	0.909
Quantile tracking	0.899	0.881	0.883	0.903
ADITI (+cap)	0.898	**0.897**	**0.903**	0.898

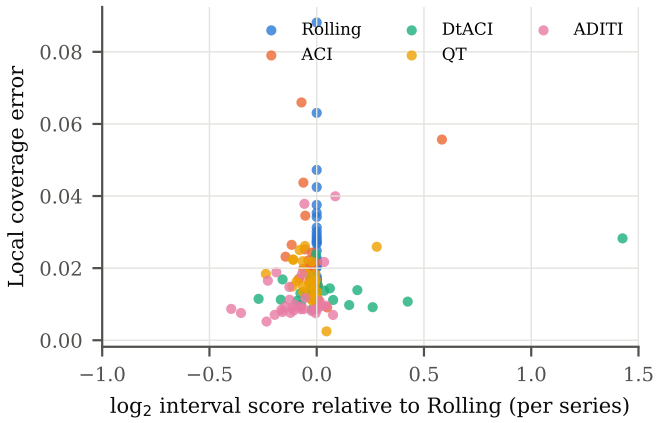


Figure 9. Per-series interval score (log ratio to Rolling) against local coverage error; lower left is better. ADITI shown with the cap.

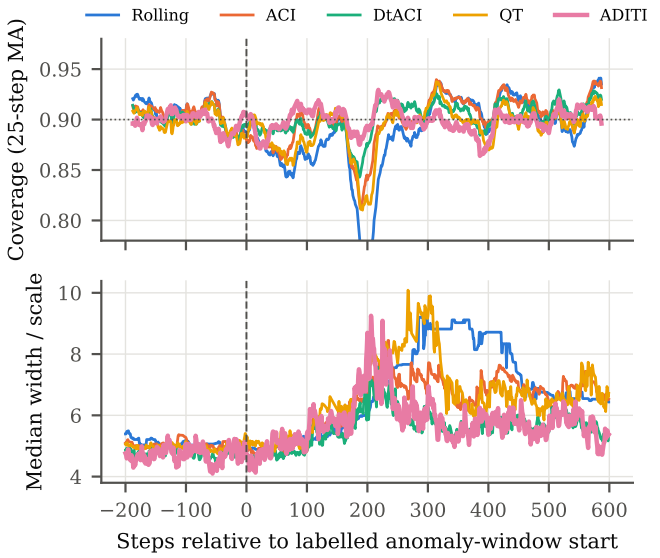


Figure 10. Coverage (top, 25-step moving average) and median normalized width (bottom) aligned at the start of 55 labelled anomaly windows (test series, AR forecaster).

incident regime and briefly under-covers the return to normal, which the slow baselines, still carrying pre-incident scores, do not. Recovery after a window ends (steps until 50-step coverage is back at 0.85) takes 2.0 steps on average for ADITI, against 1.5–3.0 for the others; all methods recover quickly.

Case studies: Fig. 11 and Fig. 12 show ADITI and two baselines in residual space during two incidents. On `machine_temperature_system_failure`, the residual jumps by an order of magnitude near step 3990 as the machine fails; all three thresholds rise with it, and ADITI's tracks the residual level more closely before and after the spike. On `Twitter_volume_AMZN`, a burst of volume spikes inside the labelled window produces repeated large residuals; ADITI widens with each burst and narrows between them, while ACI's threshold moves more slowly. These are illustrations; the event

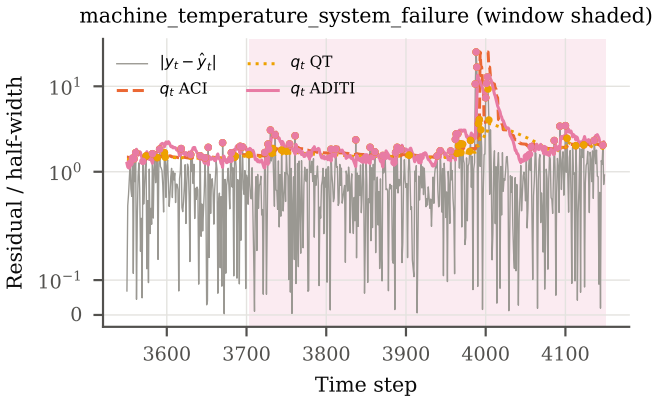


Figure 11. Residuals and half-widths on `machine_temperature_system_failure` around a labelled failure (shaded). Dots mark misses.

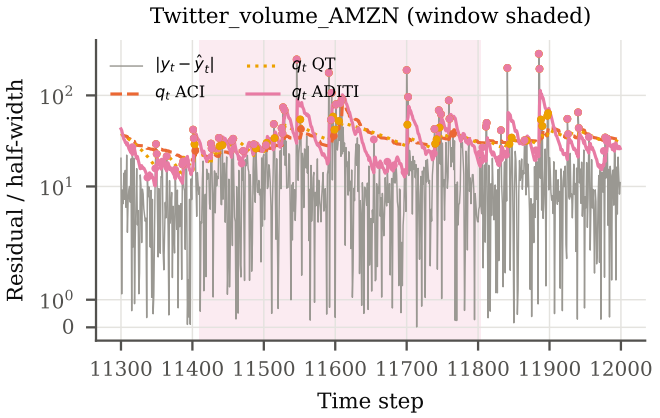


Figure 12. Residuals and half-widths on `Twitter_volume_AMZN` around a labelled window (shaded).

study above is the aggregate evidence.

C. RQ3: Ablation

Table VIII removes one ingredient at a time (Fig. 13). **Normalization** carries most of the local-coverage gain with the AR forecaster: without it, local error rises from 0.0121 to 0.0192 and coverage inside windows falls from 0.901 to 0.881. **Feedback** is what keeps marginal coverage honest: without it the mean coverage gap rises from 0.002 to 0.020 and one series misses its target by 0.218, as Theorem 1 would lead one to expect. **The drift detector** matters less than we expected. With the AR forecaster, removing it changes nothing measurable (local error 0.0121 both ways). With the naive forecaster it helps: under-coverage windows rise from 0.00% to 0.85% and mean recovery time from 1.8 to 5.6 steps without it.

D. RQ4: Robustness

Forecaster: With the naive forecaster (Table X) ADITI's advantage narrows. It still has the best coverage inside windows (0.901, better than every baseline on 23–25 of 31 series, $p \leq 0.0034$), no under-coverage windows and the best worst-case local coverage (0.862). But quantile tracking now has

Table VIII
ABLATION ON TEST SERIES

	AR forecaster			Naive forecaster		
Variant	Local	<80%	In win.	Local	<80%	Recov.
ADITI	0.0121	0.22%	0.901	0.0150	0.00%	1.8
w/o normalization	0.0192	0.01%	0.881	0.0166	0.00%	1.9
w/o drift detector	0.0121	0.31%	0.903	0.0160	0.85%	5.6
w/o feedback	0.0284	4.09%	0.890	0.0216	0.49%	4.3
normalization only	0.0273	2.94%	0.894	0.0211	0.49%	4.8

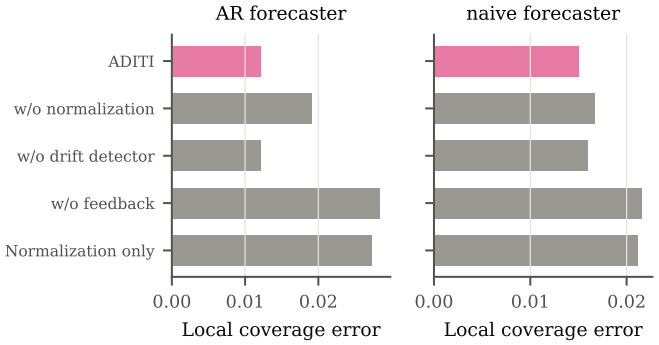


Figure 13. Ablation: local coverage error for each variant and forecaster.

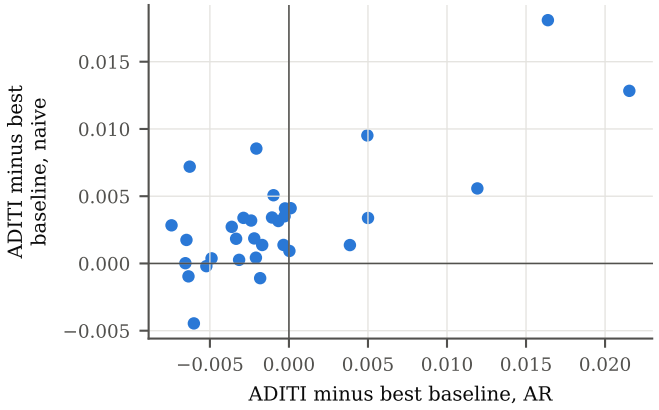


Figure 14. Per-series local coverage error of ADITI minus that of the best baseline on the same series, AR versus naive forecaster. Negative is better for ADITI.

lower local coverage error (0.0134 against 0.0150; QT better on 24 of 32 series, $p = 0.006$), and ADITI's interval score is not significantly different from rolling, ACI or DtACI, and worse than quantile tracking (ADITI better on 9 of 32 series, $p = 0.014$). Naive residuals are spiky, and a scale estimate driven by them is noisier. Fig. 14 shows that the reversal is broad rather than driven by a few series: ADITI beats the best baseline's local error on 24 series with the AR forecaster but on only 4 with the naive one, and the per-series margins are correlated across forecasters (correlation 0.74), so the series where ADITI is weakest with AR are also where it loses most with naive.

The event study tells the same story as with the AR forecaster, only compressed (Table IX): in the first 300 steps of a window, ADITI covers 0.903–0.904, DtACI 0.895–0.897 and the others 0.877–0.892.

Table IX
COVERAGE ALIGNED AT ANOMALY-WINDOW START, NAIVE FORECASTER (55 WINDOWS)

Method	$[-200, 0)$	$[0, 100)$	$[100, 300)$	$[300, 600)$
Rolling split	0.908	0.880	0.877	0.915
ACI	0.901	0.887	0.888	0.911
DtACI	0.902	0.897	0.895	0.909
Quantile tracking	0.901	0.892	0.888	0.903
ADITI (+cap)	0.906	0.903	0.904	0.909

Table X
TEST RESULTS WITH THE NAIVE FORECASTER

Method	Cov.	Local	<80%	Score (gm)	In win.
Rolling split	0.904	0.0277	0.95%	1.000	0.874
ACI	0.901	0.0204	0.95%	0.987	0.882
DtACI	0.901	0.0150	0.00%	1.044	0.888
Quantile tracking	0.899	**0.0134**	0.84%	**0.982**	0.885
ADITI	0.906	0.0150	**0.00%**	0.996	**0.901**
ADITI + cap	0.906	0.0150	**0.00%**	0.992	**0.901**

Table XI
LOCAL COVERAGE ERROR / COVERAGE IN WINDOWS AT THREE TARGETS (AR)

Method	95%	90%	80%
Rolling split	0.0181 / 0.916	0.0280 / 0.848	0.0411 / 0.746
ACI	0.0131 / 0.934	0.0209 / 0.871	0.0317 / 0.759
DtACI	0.0111 / 0.938	0.0144 / 0.885	0.0185 / 0.781
Quantile tracking	0.0172 / 0.916	0.0162 / 0.868	**0.0141** / 0.776
ADITI	**0.0083 / 0.947**	**0.0121 / 0.901**	0.0186 / **0.813**

Coverage level: Fig. 16 and Table XI repeat the evaluation at 95% and 80% targets. At 95%, ADITI's advantage grows: local error 0.0083 against 0.0111 for DtACI and 0.0131 for ACI, and coverage inside windows 0.947 against 0.916–0.938. At 80%, it disappears: quantile tracking has the lowest local error (0.0141) and DtACI and ADITI follow (0.0185, 0.0186); ADITI still has the highest coverage inside windows (0.813). The weighted quantile of normalized scores is most useful in the upper tail, where a few large residuals decide the band. Fig. 15 puts the same results on a width–steadiness plane: at 95% and 90% ADITI sits in the lower left, narrowest and steadiest together, while at 80% quantile tracking is steadier and DtACI narrower.

Data category: Fig. 17 and Table XII split the AR results by category. ADITI has the lowest local error on AWS (0.0143), traffic (0.0086) and tweet series (0.0088); DtACI is lower on ad-exchange (0.0120 against 0.0130) and quantile tracking on known-cause series (0.0145 against 0.0147). Its interval score is best on ad-exchange, traffic and tweets, and worst on AWS and known-cause series, where the two failure series sit.

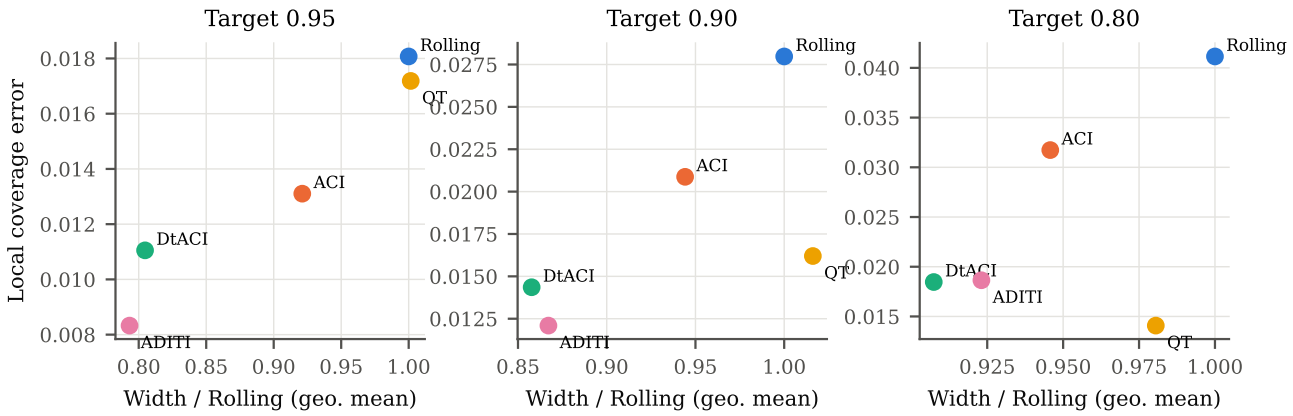


Figure 15. Band width (geometric mean relative to Rolling) against local coverage error at three targets (AR forecaster). Lower left is better.

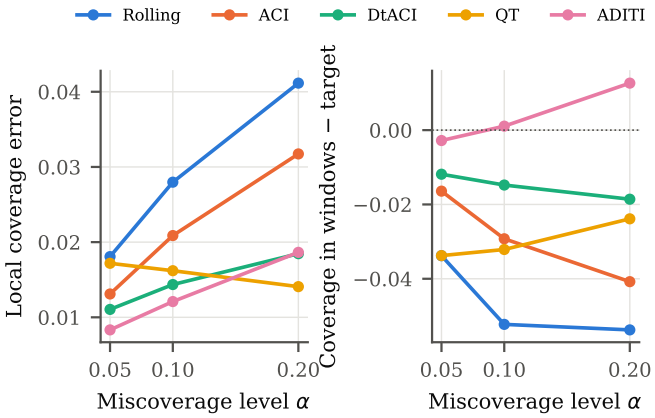


Figure 16. Local coverage error (left) and coverage in anomaly windows relative to target (right) versus α.

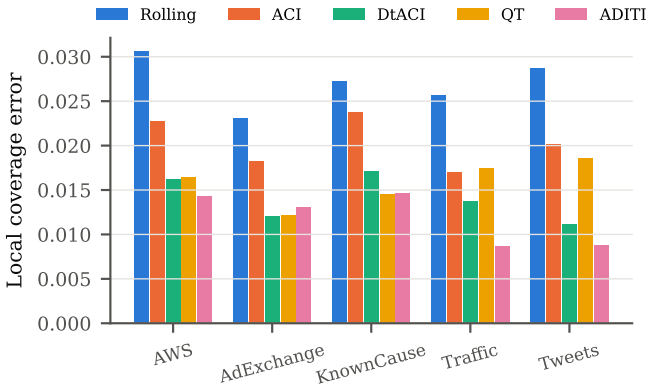


Figure 17. Local coverage error by data category (AR forecaster).

Table XII
BY CATEGORY (AR): LOCAL ERROR / INTERVAL SCORE (GM VS. ROLLING)

Category	ACI	DtACI	QT	ADITI†
AWS (11)	.0228 / 1.02	.0162 / 1.28	.0164 / 0.99	**.0143** / 8.09
Ad exch. (4)	.0182 / 0.99	**.0120** / 0.99	.0122 / 0.98	.0130 / **0.94**
Known cause (5)	.0237 / 0.99	.0171 / 1.20	**.0145** / **0.96**	.0147 / 1.36
Traffic (5)	.0171 / 0.99	.0138 / 0.99	.0175 / 0.98	**.0086** / **0.93**
Tweets (7)	.0201 / 0.97	.0112 / 1.34	.0185 / 0.96	**.0088** / **0.90**

†As selected (no cap); AWS and known-cause each contain one failure series.

Hyperparameter sensitivity: Fig. 18 varies each ADITI hyperparameter around its selected value on the test series (one at a time, cap on; reported, not used for selection). Local coverage error stays between 0.0074 and 0.0162 across the 21 distinct settings, at or below DtACI's 0.0144 in all but two ($\gamma = 0.002$ and $\beta = 0.02$). Interval scores in this study are relative to rolling with $W = 500$, the development reference, so the selected setting scores 0.988 rather than the 0.995 of Table V; Appendix D lists every value. The feedback step γ is a direct trade-off: $\gamma = 0.02$ gives local error 0.0074 at an interval score of 1.053, and $\gamma = 0.002$ gives 0.0162 at 0.970. A fast forgetting factor that is too aggressive ($\lambda_f = 0.9$, effective size 19) raises the interval score to 1.19; $\lambda_f \geq 0.98$ is safe. The CUSUM threshold h hardly matters between 5 and 20, consistent with the small ablation effect of the detector.

Development versus test: Fig. 19 compares each method's local coverage error on development and test series. The ranking is identical on both, and every method does somewhat better on test than on development, so selection did not overfit the development set in ADITI's favour.

Drift alarms: On the 32 test series ADITI raised 156 alarms (0.68 per 1000 steps) on 15 series; 39 of them (25%) fell inside a labelled anomaly window or within 50 steps before one. The detector is therefore not an anomaly detector: three quarters of its alarms mark volatility changes that NAB does not label. That is by design, since any change in residual scale should trigger faster forgetting, labelled or not.

Runtime: Fig. 20 reports median per-step cost. ADITI takes 67 μs per step in Python, dominated by sorting 1000 stored scores, against 2–3 μs for rolling, ACI and quantile tracking and 18 μs for DtACI. At NAB's 5-minute sampling interval

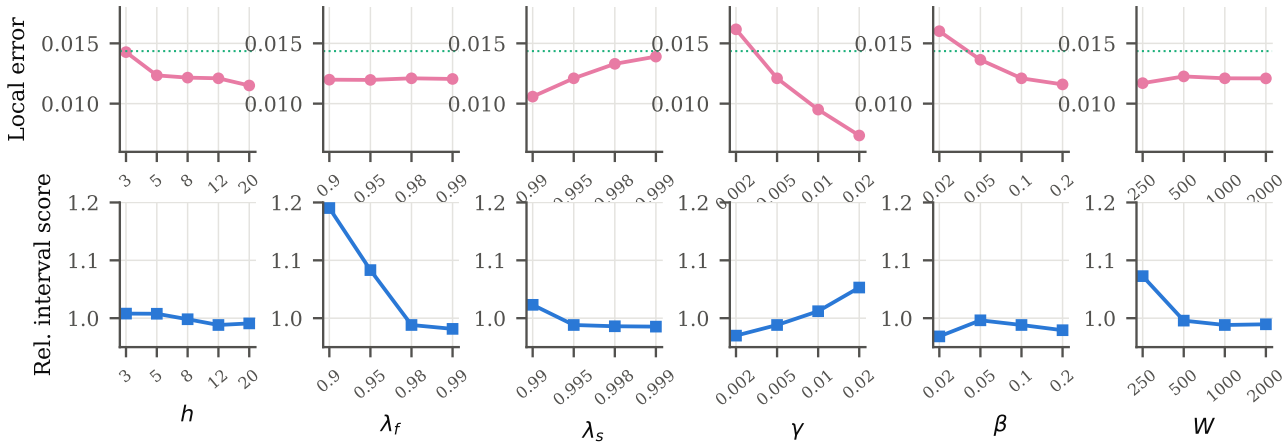


Figure 18. One-at-a-time sensitivity on test series: local coverage error (top; dotted: DtACI) and interval score relative to Rolling (bottom).

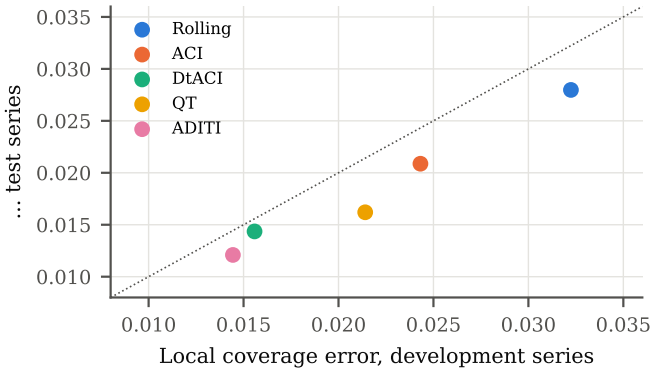


Figure 19. Local coverage error on development versus test series (AR forecaster).

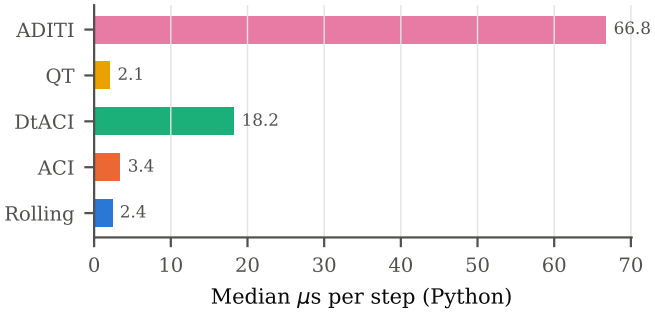


Figure 20. Median per-step cost (single-threaded Python).

this is negligible; at very high rates an order-statistics tree would bring it to $O(\log W)$.

E. Failure analysis

The as-selected ADITI failed badly on two test series. On `ec2_disk_write_bytes_1ef3de`, a disk-write counter that is near zero for long stretches and then bursts by many orders of magnitude, the mean normalized width reached 3.6×10^{14}, against 9.9 for rolling, while the median width (3.3) stayed small: a handful of steps carried astronomically wide intervals. On `rogue_agent_key_updown`, the mean width was 148 against 9.3. Both are instances of Proposition 3: a burst arrived while $\hat{\sigma}$ was tiny, left a huge normalized score in memory, and was later multiplied by a large $\hat{\sigma}$. The cap (10) removes the multiplication. On development series it changed nothing (geometric-mean interval score 0.9419 versus 0.9436 with AR); on test series it brought ADITI's geometric mean from 2.06 to 0.995 without changing coverage, local error or the median. After the cap, `rogue_agent_key_updown` is at parity with rolling (ratio 1.06), but `ec2_disk_write_bytes_1ef3de` still has a 6.3× interval-score ratio: its residuals jump by orders of magnitude within the window used for the cap. ACI scores worse still on the same series (21.6×), but that does not excuse ADITI's failure there. We report the cap as post-hoc, because it was designed after seeing the test failure.

VIII. Controlled Experiments

NAB tells us how ADITI behaves on real streams but not why, because the changes in its series are unlabelled except for anomaly windows. To separate the mechanisms we generated streams with one known change at step 3000 of 6000 and ran every method with the settings frozen for the AR forecaster (Table IV), on the residuals directly ($\hat{y}_t = 0$), 40 independent runs per scenario. Nothing here was tuned; the settings were fixed before the synthetic streams were generated.

A. Scenarios

Residuals are $s_t = |\sigma_t \varepsilon_t|$ with ε_t standard Gaussian unless stated.

- *Scale jump*: σ_t steps from 1 to 4.
- *Calming*: σ_t steps from 1 to 0.25.
- *Shape change*: ε_t switches to a Student t_2 variable scaled to the same median absolute value, so the scale is unchanged and only the tail grows.
- *Gradual drift*: σ_t rises linearly from 1 to 4 over 1500 steps.
- *Bursts*: five episodes of 100 steps with $\sigma_t = 5$, 600 steps

Table XIII

CONTROLLED EXPERIMENTS (40 RUNS EACH): LOCAL ERROR / COVERAGE IN CHANGE WINDOW / INTERVAL SCORE VS. ROLLING

Method	Scale	Calming	Shape	Gradual	Bursts
	0.0199	0.0160	0.0138	0.0200	0.0478
	0.752	0.977	0.865	0.861	0.501
Rolling	1.00	1.00	1.00	1.00	1.00
	0.0112	0.0162	0.0104	0.0099	0.0300
	0.883	0.981	0.884	0.897	0.753
ACI	0.99	1.00	1.01	1.00	0.84
	0.0127	0.0117	0.0120	0.0128	0.0142
	0.867	0.919	0.869	0.890	0.858
DtACI	0.99	0.99	1.00	1.00	0.78
	0.0184	0.0172	0.0102	0.0206	0.0462
	0.746	1.000	0.871	0.854	0.545
QT	1.00	1.01	1.01	1.00	0.92
	0.0081	**0.0080**	**0.0075**	**0.0079**	**0.0089**
	0.902	0.901	0.896	0.900	0.884
ADITI	1.03	1.02	1.06	1.04	0.73
	0.0081	0.0080	0.0076	0.0079	0.0090
	0.902	0.900	0.896	0.900	0.884
w/o detector	1.03	1.02	1.06	1.04	0.73
	0.0117	0.0132	0.0109	0.0110	0.0215
	0.864	0.947	0.880	0.896	0.771
w/o norm.	0.99	0.99	1.00	1.00	0.79

Each cell: local coverage error (top), coverage in the change window, interval score.

apart.

Coverage "in window" is measured over the 300 steps after the change (the 1500-step ramp for gradual drift, the bursts themselves for bursts).

B. Results

Fig. 21 shows coverage around the change and Table XIII the summary. ADITI has the lowest local coverage error in all five scenarios and its coverage in the change window is the closest to target in all five. The differences are largest where the scale moves fast. After a scale jump, smoothed coverage of the rolling method falls to 0.35 and that of every baseline below 0.52; ADITI's falls to 0.76 for a few steps and is back at target about 25 steps after the change, the length of the smoothing window. During bursts, rolling, ACI and quantile tracking cover about half of the burst observations (0.50–0.75), DtACI 0.86 and ADITI 0.88. Calming exposes a different failure of feedback-only methods: ACI first over-covers, drives α_t up to compensate, and then under-covers (0.78) about 500 steps after the change; ADITI rescales within a few steps and never leaves the 0.87–0.99 band.

The price is efficiency when nothing is wrong. In the four scenarios where the stream is i.i.d. apart from one change, ADITI's interval score is 2–6% above rolling, because normalizing by a noisy scale estimate widens the band slightly on average. In the burst scenario, where scale changes repeatedly, its interval score is 27% below rolling and the lowest of all methods (Fig. 22). Operational metrics look much more like the burst scenario than like the i.i.d. ones, which is consistent with ADITI's lower median interval score on NAB.

Table XIV

DETECTOR THRESHOLD ON CONTROLLED STREAMS (20 RUNS): LOCAL ERROR / COVERAGE IN WINDOW / ALARMS PER RUN

h	Shape change	Calming	Bursts
3	0.0100 / 0.906 / 103.1	0.0122 / 0.914 / 97.8	0.0124 / 0.905 / 107.4
5	0.0079 / 0.902 / 23.3	0.0085 / 0.907 / 24.1	0.0094 / 0.897 / 29.1
12	0.0075 / 0.896 / 0.8	0.0080 / 0.903 / 0.2	0.0091 / 0.885 / 1.4

C. What the detector does

The two ablation rows in Table XIII answer the question the NAB ablation left open. Removing normalization costs ADITI its advantage in every scenario. Removing the drift detector changes nothing measurable, including in the shape-change scenario that we designed to need it. The reason is visible in the alarm counts: with the selected threshold $h = 12$ the detector fired on average 0.8 times per run under the shape change, 0.2 under calming and 1.4 under bursts, and in ten runs per scenario that we inspected it never fired before the change. Normalization absorbs scale changes so quickly that the standardized log score barely moves, and the CUSUM sum rarely accumulates past the threshold. Lowering the threshold makes the detector active but not useful (Table XIV, 20 runs): at $h = 5$ it fires 23–29 times per run, mostly on noise, and local error is no better; at $h = 3$ it fires about 100 times per run and local error rises by a third to a half, because the fast memory then shrinks the calibration set to about 100 scores most of the time. On NAB the detector fired 0.68 times per 1000 steps; its only measurable benefit was with the naive forecaster (Section VII-C).

IX. DISCUSSION

A. What the results say

Table XV condenses the evidence for each research question. The clearest result is RQ2: when an incident starts, ADITI keeps coverage at the target while every baseline dips, and it does so on most series, not on average over a few. RQ1 is a qualified yes: ADITI is the steadiest method and the most efficient on the typical series, but the method as selected had a severe failure mode that a cap, added after the test run, was needed to remove. RQ3 has a less tidy answer than the design suggests: normalization and feedback do the work with a good forecaster, and the drift detector earns its place mainly with a poor one. RQ4 marks the limits: a naive forecaster, a low coverage target and two heavy-tailed series.

B. Why normalization helps most

Proposition 2 gives the mechanism. On these streams, the residual scale changes faster than any calibration window can follow: a 5-minute CPU metric can go from quiet to busy within an hour. A method that calibrates raw residuals must see misses before it widens, and must see many hits before it narrows. A method that divides by a running scale widens on the first large residual, because the next $\hat{\sigma}$ is already larger. The weighted quantile then only has to track the *shape* of the normalized distribution, which changes much more slowly than

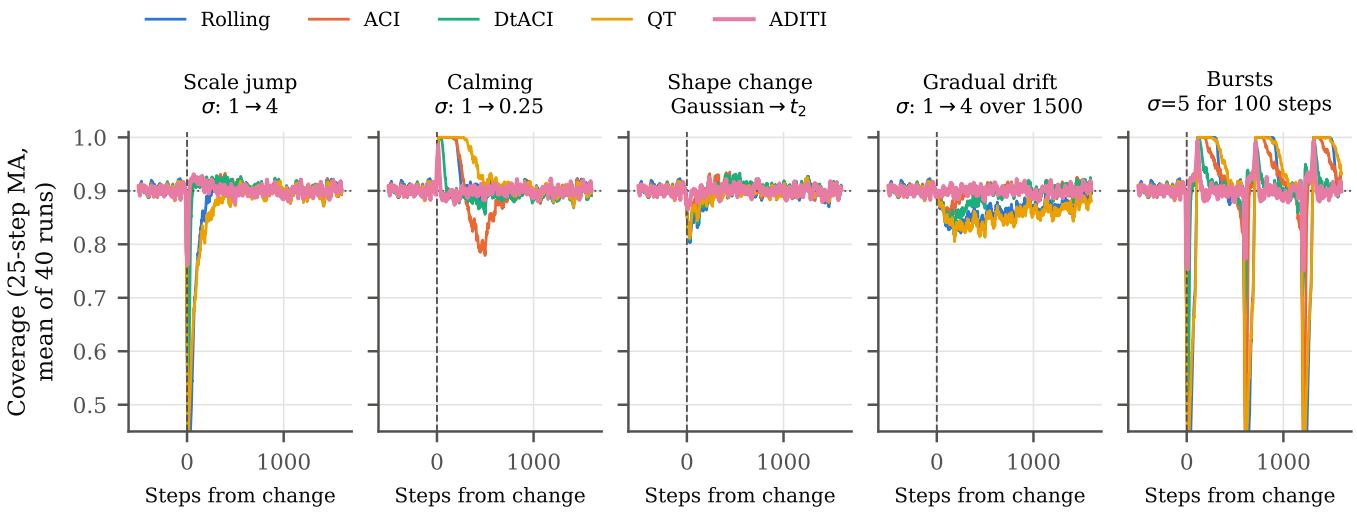


Figure 21. Controlled experiments: coverage around a known change (25-step moving average of the mean over 40 runs; dotted line: target 0.9).

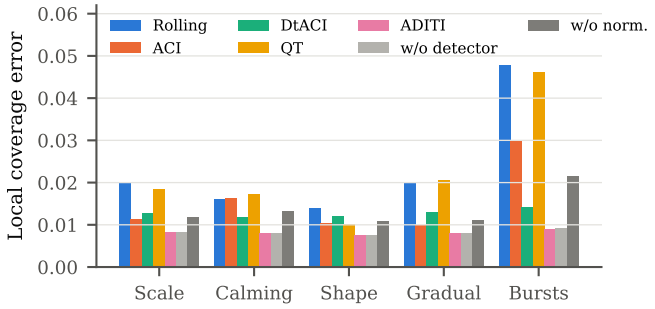


Figure 22. Local coverage error by scenario, including the two ADITI ablations.

Table XV
SUMMARY OF FINDINGS

RQ	Verdict	Evidence
RQ1	Yes, with a fix	Lowest local error (31/32 vs. rolling, 25/32 vs. DtACI); lowest median interval score; 2 series need the post-hoc cap
RQ2	Yes	Only method at target inside windows (0.901); no dip after window start
RQ3	Partly	Normalization and feedback carry the gain on NAB and on controlled streams; detector helps only with naive forecaster (recovery 1.8 vs. 5.6)
RQ4	Mixed	Gains grow at 95%, vanish at 80%; QT steadier with naive forecaster

its scale. This also explains why ADITI's advantage grows at 95% coverage and fades at 80%. The 95% quantile depends on a few large residuals, exactly the ones whose size depends on the current scale. The 80% quantile sits in the bulk of the distribution, where raw quantile tracking with a well-tuned step is already accurate.

C. *Why the detector mattered less*

We designed ADITI around the drift detector, so the ablation result surprised us. With the AR forecaster, removing the detector leaves local error unchanged (0.0121). The controlled experiments (Section VIII) explain why: normalization absorbs a change in volatility within a few steps, so the standardized log score that the CUSUM watches barely moves, and at the selected threshold the detector almost never fires, even under a pure shape change that we built for it. Lower thresholds make it fire on noise and hurt. With the naive forecaster, residuals carry more of the series' own dynamics (a level shift appears as a run of large residuals), and there the detector's faster memory did shorten recovery (1.8 versus 5.6 steps). Our reading is that, once residuals are normalized, a mean-shift CUSUM on their logarithm is the wrong detector; Section X sketches a better one. Only a quarter of alarms fall near labelled windows (Section VII-D.6); ADITI should not be used as an anomaly detector.

D. *When to use something else*

Our results point to three situations where a simpler method is the better choice.

- *Weak forecaster, steadiness the priority.* With the naive forecaster quantile tracking had lower local error and a lower interval score than ADITI; it is also 30× cheaper per step.
- *Low coverage targets.* At 80% coverage quantile tracking and DtACI were steadier, and ADITI's recovery time was the worst (80 steps against 32–73).
- *Counters with long flat stretches and bursts.* Disk-write counters and similar series produce the width blow-up of Proposition 3. The cap bounds it, but one such series still costs 6.3× the rolling interval score.

ADITI is the better choice when the target is 90% or higher, the forecaster is reasonable and the intervals matter most during

incidents, which is the common case for alerting.

E. Deployment guidance

- Always enable the cap (10); it changed nothing on development data and removes the worst failure.
- Start from $\lambda_s = 0.995$, $\lambda_f = 0.98$, $h = 12$, $\beta = 0.1$; Fig. 18 shows wide flat regions around them. Avoid $\lambda_f < 0.95$.
- Choose γ by the trade-off you want: 0.005 for efficiency, 0.01–0.02 for steadier coverage at a few percent higher interval score.
- With a naive or poor forecaster, use the winsorized scale ($\kappa = 4$) and a slower scale rate ($\beta = 0.02$).
- Monitor the share of capped steps: a high share signals a series of the kind ADITI handles badly.

F. Threats to validity

Internal. The cap was designed after seeing the test failure; we report the method with and without it and checked the cap on development data before re-running test series, but the capped results are not held-out in the strict sense. Hyperparameters were selected by one criterion (interval score subject to coverage); a criterion that weights local coverage would select differently, likely in ADITI's favour on local error and against it on efficiency. Baselines were tuned on development data with their own grids (Table IV), which were smaller than ADITI's; DtACI has no tuning beyond its window, following its authors' recommendations.

External. NAB is one benchmark of 47 real series, sampled mostly at 5-minute intervals. Its labelled windows are drawn generously around incidents and include stretches of normal behaviour, which dilutes the event-study contrasts. We used two simple forecasters; with a strong learned forecaster, residuals would be closer to white noise and all methods would differ less.

Construct. Local coverage error over 250-step windows is our main steadiness metric; we did not vary the window length, and a different length would change the values. The interval score charges infinite intervals at the largest finite width, which penalizes DtACI's 1.05% infinite intervals heavily; its median interval score (0.992) is the fairer summary for DtACI.

Statistical. Thirty-two series give modest power; we report per-series counts alongside p-values and do not correct for multiple comparisons. Under a Bonferroni correction over the 16 AR tests in Table VI (threshold 0.0031), all four in-window results and three of the four local-error results survive; local error against DtACI ($p = 0.0032$) narrowly misses, as do the interval-score comparisons with DtACI and quantile tracking.

X. Conclusion

ADITI combines three known ideas, scale normalization, drift-triggered short memory and coverage feedback, into an online conformal method for operational time series. On 32 held-out NAB series it held 90% coverage more steadily than rolling, ACI, DtACI and quantile tracking, was the only method at target inside labelled anomaly windows, and gave the lowest median interval score. The evaluation also found where it fails: a multiplicative width blow-up on two heavy-tailed series, repaired only after the fact by a one-line cap; a smaller or reversed advantage with a naive forecaster and at an 80% target; and a drift detector that contributes less than its central place in the design suggests. Controlled experiments with known changes confirm the first two ingredients and show why the third rarely acts: once residuals are normalized, a mean-shift CUSUM on their logarithm seldom crosses its threshold. Two directions follow. The first is a principled version of the cap: a bound on the ratio $\hat{\sigma}_t / \hat{\sigma}_i$ for stored scores, which would keep Proposition 3's factor bounded by construction. The second is a detector matched to what normalization leaves behind: a test on the tail of the normalized scores (for example, the exceedance rate of a high normalized quantile) rather than a mean-shift CUSUM on their logarithm, which our controlled experiments show rarely fires.

Reproducibility

All code, the NAB commit hash, the development/test split, the frozen hyperparameters and the raw per-series results are in the accompanying source archive. The analysis and plotting scripts regenerate every table and figure from the stored results. No number in this paper was produced outside that pipeline.

References

[1] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York, NY, USA: Springer, 2005.

[2] J. Lei, M. G'Sell, A. Rinaldo, R. J. Tibshirani, and L. Wasserman, "Distribution-free predictive inference for regression," *J. Amer. Statist. Assoc.*, vol. 113, no. 523, pp. 1094–1111, 2018.

[3] G. Shafer and V. Vovk, "A tutorial on conformal prediction," *J. Mach. Learn. Res.*, vol. 9, pp. 371–421, 2008.

[4] A. N. Angelopoulos and S. Bates, "Conformal prediction: A gentle introduction," *Found. Trends Mach. Learn.*, vol. 16, no. 4, pp. 494–591, 2023.

[5] I. Gibbs and E. Candès, "Adaptive conformal inference under distribution shift," in *Proc. NeurIPS*, 2021, pp. 1660–1672.

[6] I. Gibbs and E. J. Candès, "Conformal inference for online prediction with arbitrary distribution shifts," *J. Mach. Learn. Res.*, vol. 25, no. 162, pp. 1–36, 2024.

[7] A. N. Angelopoulos, E. J. Candès, and R. J. Tibshirani, "Conformal PID control for time series prediction," in *Proc. NeurIPS*, 2023.

[8] A. Bhatnagar, H. Wang, C. Xiong, and Y. Bai, "Improved online conformal prediction via strongly adaptive online learning," in *Proc. ICML*, PMLR 202, 2023, pp. 2337–2363.

[9] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.

[10] A. Lavin and S. Ahmad, "Evaluating real-time anomaly detection algorithms: The Numenta Anomaly Benchmark," in *Proc. IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, 2015, pp. 38–44.

[11] H. Papadopoulos, K. Proedrou, V. Vovk, and A. Gammerman, "Inductive confidence machines for regression," in *Proc. ECML*, LNCS 2430, 2002, pp. 345–356.

[12] Y. Romano, E. Patterson, and E. Candès, "Conformalized quantile regression," in *Proc. NeurIPS*, 2019.

[13] R. J. Tibshirani, R. Foygel Barber, E. Candès, and A. Ramdas, "Conformal prediction under covariate shift," in *Proc. NeurIPS*, 2019.

[14] R. Foygel Barber, E. J. Candès, A. Ramdas, and R. J. Tibshirani, "Conformal prediction beyond exchangeability," *Ann. Statist.*, vol. 51, no. 2, pp. 816–845, 2023.

[15] M. Zaffran, O. Féron, Y. Goude, J. Josse, and A. Dieuleveut, "Adaptive conformal predictions for time series," in *Proc. ICML*, PMLR 162, 2022, pp. 25834–25866.

[16] C. Xu and Y. Xie, "Conformal prediction interval for dynamic time-series," in *Proc. ICML*, PMLR 139, 2021, pp. 11559–11569.

[17] C. Xu and Y. Xie, "Sequential predictive conformal inference for time series," in *Proc. ICML*, PMLR 202, 2023, pp. 38707–38727.

[18] K. Stankevičiūtė, A. M. Alaa, and M. van der Schaar, "Conformal time-series forecasting," in *Proc. NeurIPS*, 2021.

[19] M. Basseville and I. V. Nikiforov, *Detection of Abrupt Changes: Theory and Application*. Englewood Cliffs, NJ, USA: Prentice Hall, 1993.

[20] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surv.*, vol. 46, no. 4, Art. 44, 2014.

[21] S. Ahmad, A. Lavin, S. Purdy, and Z. Agha, "Unsupervised real-time anomaly detection for streaming data," *Neurocomputing*, vol. 262, pp. 134–147, 2017.

[22] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.

[23] R. L. Winkler, "A decision-theoretic approach to interval estimation," *J. Amer. Statist. Assoc.*, vol. 67, no. 337, pp. 187–191, 1972.

[24] T. Gneiting and A. E. Raftery, "Strictly proper scoring rules, prediction, and estimation," *J. Amer. Statist. Assoc.*, vol. 102, no. 477, pp. 359–378, 2007.

[25] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bull.*, vol. 1, no. 6, pp. 80–83, 1945.

Appendix A: Per-Series Test Results

Table XVI lists every test series with the AR forecaster at 90% target. ADITI has the lowest local coverage error on 24 of 32 series, quantile tracking on 5 and DtACI on 3. The interval-score columns show where each method is efficient: values below 1 beat the rolling baseline. The "after" column is ADITI's coverage in the 500 steps following each labelled window (Section VII); a dash marks series without labelled windows that qualify.

Appendix B: Development Results and Grids

Table XVII gives each method's selected setting on the 15 development series. Two features matter for reading the test results. First, ADITI was already the steadiest method with the AR forecaster on development data (local error 0.0144, against 0.0156 for DtACI), so the test result was predicted rather than found. Second, ADITI's weakness with the naive forecaster was also visible: its interval score (1.015) was the worst of the five, and quantile tracking and DtACI had lower local error. The second development round, which added the winsorized scale, narrowed that gap (from 1.049 in round 1) but did not close it.

Grids. Rolling: $W \in \{250, 500, 1000\}$. ACI: $\gamma \in \{0.002, 0.005, 0.01, 0.02\} \times W \in \{500, 1000\}$. DtACI: $W \in \{500, 1000\}$. Quantile tracking: $\eta \in \{0.01, 0.03, 0.1, 0.3\}$. ADITI round 1 (36 settings per forecaster, $\gamma = 0.005$, no clipping): $\lambda_s \in \{0.995, 0.998\}$, $\lambda_f \in \{0.95, 0.98\}$, $h \in \{5, 8, 12\}$, $\beta \in \{0.02, 0.05, 0.1\}$. ADITI round 2 (16 settings per forecaster, $\lambda_s = 0.995$, $\lambda_f = 0.98$ from round 1): $h \in \{5, 12\}$, $\beta \in \{0.02, 0.1\}$, $\gamma \in \{0.005, 0.01\}$, $\kappa \in \{\text{none}, 4\}$. Across the 16 round-2 settings with the AR forecaster, ADITI's development local error ranged from 0.0103 to 0.0192 and its interval score from 0.942 to 1.024, so the choice of setting matters less than the choice of method.

Appendix C: Metric Details

Local coverage error is the mean absolute deviation of 250-step rolling coverage from $1 - \alpha$, computed after a 300-step warm-up. *Worst local* is the minimum of that rolling coverage over the series. *Interval score* [23,24] is $2u_t + \frac{2}{\alpha}(|y_t - \hat{y}_t| - u_t)_+$ averaged over the series; ratios to rolling are taken per series and combined by geometric mean. *Coverage in windows* is the coverage within each labelled window (after warm-up), averaged over the windows of a series and then over series. *After* is the same over the 500 steps following each window end. *Recovery* is the number of steps after a window ends until 50-step rolling coverage first reaches 0.85 (zero if it never fell below). Widths and interval scores are divided by each series' median absolute residual under the same forecaster before averaging across series, so that series on very different scales can be combined.

Appendix D: Sensitivity Values

Table XVIII gives the values behind Fig. 18. Each row changes one hyperparameter of the selected AR setting (marked *) and keeps the cap on. Interval scores are relative to rolling with $W = 500$, the reference used during development.

Table XVI

PER-SERIES TEST RESULTS (AR FORECASTER, $1-\alpha=0.9$)

			Local coverage error ↓					Interval score / Rolling ↓				ADITI coverage	
Series	Cat.	n	Roll.	ACI	DtACI	QT	ADITI	ACI	DtACI	QT	ADITI+cap	in win.	after
ec2_cpu_utilization_fe7f93	AWS	4,032	0.038	0.026	0.011	0.022	**0.011**	0.92	0.83	0.93	0.78	0.898	0.901
ec2_cpu_utilization_ac20cd	AWS	4,032	0.023	0.017	**0.011**	0.015	0.015	0.94	0.94	0.96	0.92	0.918	0.852
rds_cpu_utilization_e47b3b	AWS	4,032	0.029	0.019	0.018	0.016	**0.009**	0.99	0.98	0.98	0.88	0.897	0.893
elb_request_count_8c0756	AWS	4,032	0.018	0.015	0.012	0.014	**0.007**	0.99	0.99	0.99	1.01	0.885	0.902
grok_asg_anomaly	AWS	4,621	0.025	0.017	0.014	0.022	**0.011**	0.98	1.04	0.98	0.96	0.892	0.905
ec2_cpu_utilization_c6585a	AWS	4,032	0.017	0.009	0.012	**0.002**	0.019	1.03	0.99	1.03	1.02	–	–
ec2_disk_write_bytes_c0d644	AWS	4,032	0.042	0.044	0.024	0.025	**0.022**	0.96	1.00	0.95	0.87	0.893	0.903
iio_NetworkIn	AWS	1,243	0.027	0.025	0.011	0.016	**0.011**	0.96	0.89	0.94	0.92	0.952	0.900
ec2_disk_write_bytes_1ef3de	AWS	4,730	0.088	0.056	0.040	**0.026**	0.038	1.50	21.55	1.21	6.29	0.881	0.834
ec2_cpu_utilization_53ea38	AWS	4,032	0.016	0.013	0.013	0.010	**0.008**	1.00	0.99	1.00	1.00	0.887	0.903
rds_cpu_utilization_cc0c53	AWS	4,032	0.014	0.010	0.012	0.011	**0.007**	1.00	1.00	0.99	0.99	0.905	0.904
exchange-3_cpc_results	Ad	1,538	0.022	0.016	0.012	**0.012**	0.017	0.99	0.97	0.98	0.95	0.924	0.895
exchange-4_cpm_results	Ad	1,643	0.034	0.024	0.014	**0.013**	0.018	0.99	1.02	0.96	0.85	0.919	0.891
exchange-2_cpm_results	Ad	1,624	0.015	0.011	**0.009**	0.012	0.009	0.99	0.99	0.99	1.00	0.894	0.897
exchange-3_cpm_results	Ad	1,538	0.021	0.022	0.014	0.012	**0.009**	1.00	0.98	0.99	0.95	0.914	0.883
machine_temperature_system_failure	KC	22,695	0.020	0.015	0.015	0.016	**0.008**	1.00	0.99	0.99	1.02	0.897	0.903
ambient_temperature_system_failure	KC	7,267	0.016	0.011	0.015	0.012	**0.010**	1.00	1.00	1.00	1.05	0.892	0.897
cpu_utilization_asg_misconfiguration	KC	18,050	0.021	0.013	0.013	0.014	**0.007**	0.99	0.95	0.99	0.96	0.899	–
ec2_request_latency_system_failure	KC	4,032	0.016	0.013	0.014	0.012	**0.009**	0.99	0.99	0.99	0.98	0.886	0.899
rogue_agent_key_updown	KC	5,315	0.063	0.066	0.028	**0.018**	0.040	0.95	2.69	0.85	1.06	0.926	0.921
occupancy_t4013	Tr	2,500	0.022	0.015	0.015	0.017	**0.008**	1.00	0.99	1.00	0.96	0.863	0.900
speed_t4013	Tr	2,495	0.020	0.014	0.010	0.018	**0.010**	1.00	0.95	1.00	0.93	0.927	0.892
TravelTime_387	Tr	2,500	0.047	0.035	0.014	0.022	**0.012**	0.96	1.14	0.93	0.85	0.919	0.893
speed_7578	Tr	1,127	0.022	0.010	0.017	0.016	**0.008**	1.00	0.90	0.99	0.89	0.848	0.894
speed_6005	Tr	2,500	0.017	0.012	0.013	0.013	**0.005**	1.00	1.00	1.00	1.03	0.924	–
Twitter_volume_KO	Tw	15,851	0.028	0.020	0.009	0.016	**0.009**	0.97	4.15	0.98	0.93	0.895	0.906
Twitter_volume_IBM	Tw	15,893	0.031	0.022	0.010	0.022	**0.009**	0.97	1.11	0.96	0.90	0.902	0.889
Twitter_volume_CRM	Tw	15,902	0.030	0.021	**0.009**	0.026	0.009	0.98	1.20	0.96	0.90	0.897	0.901
Twitter_volume_CVS	Tw	15,853	0.027	0.020	0.015	0.019	**0.009**	0.97	0.99	0.96	0.92	0.903	0.900
Twitter_volume_GOOG	Tw	15,842	0.028	0.020	0.012	0.018	**0.008**	0.96	0.98	0.97	0.92	0.898	0.902
Twitter_volume_AAPL	Tw	15,902	0.035	0.023	0.011	0.015	**0.010**	0.90	1.34	0.92	0.76	0.896	0.899
Twitter_volume_AMZN	Tw	15,831	0.022	0.015	0.011	0.013	**0.008**	1.00	1.05	0.98	0.95	0.901	0.898

n: series length. Categories: AWS CloudWatch, Ad exchange, KC known cause, Tr traffic, Tw tweets. Bold: lowest local error in the row. ADITI's local error and coverage are identical with and without the cap.

Table XVII

SELECTED SETTINGS ON DEVELOPMENT SERIES

	AR forecaster			Naive forecaster		
Method	Local	Score	In win.	Local	Score	In win.
Rolling ($W=250$)	0.0322	0.990	0.834	0.0294	0.991	0.874
ACI	0.0243	0.943	0.865	0.0222	0.975	0.888
DtACI	0.0156	1.239	0.890	0.0142	0.981	0.899
QT	0.0214	0.990	0.865	**0.0136**	**0.968**	0.888
ADITI	**0.0144**	**0.942**	**0.899**	0.0171	1.015	0.899

Score: geometric-mean interval score relative to Rolling with $W=500$.

Table XVIII

ONE-AT-A-TIME SENSITIVITY, AR FORECASTER, 32 TEST SERIES

Param.	Value	Local	Score	Coverage	In win.
h	3	0.0143	1.008	0.906	0.909
	5	0.0123	1.008	0.902	0.904
	8	0.0122	0.998	0.899	0.901
	12*	0.0121	0.988	0.899	0.901
	20	0.0115	0.991	0.899	0.902
λ_f	0.9	0.0120	1.190	0.901	0.904
	0.95	0.0120	1.083	0.899	0.902
	0.98*	0.0121	0.988	0.899	0.901
	0.99	0.0120	0.982	0.899	0.901
λ_s	0.99	0.0106	1.023	0.899	0.902
	0.995*	0.0121	0.988	0.899	0.901
	0.998	0.0133	0.986	0.899	0.900
	0.999	0.0139	0.985	0.899	0.900
γ	0.002	0.0162	0.970	0.897	0.898
	0.005*	0.0121	0.988	0.899	0.901
	0.01	0.0095	1.012	0.900	0.903
	0.02	0.0074	1.053	0.900	0.903
β	0.02	0.0160	0.968	0.897	0.885
	0.05	0.0136	0.996	0.898	0.896
	0.1*	0.0121	0.988	0.899	0.901
	0.2	0.0116	0.979	0.900	0.903
W	250	0.0117	1.073	0.898	0.902
	500	0.0123	0.996	0.898	0.901
	1000*	0.0121	0.988	0.899	0.901
	2000	0.0121	0.989	0.899	0.901

Appendix E: Per-Series Results with the Naive Forecaster

Table XIX repeats Table XVI for the naive forecaster. Here quantile tracking has the lowest local error on 20 series, DtACI on 8 and ADITI on 4, matching the aggregate reversal in Table X.

Appendix F: Coverage Bound Against Observation

Theorem 1 bounds ADITI's long-run coverage gap by $(N_T + 1)(1 + 2\gamma)/(\gamma T)$. Table XX evaluates the bound on the 16 test series with the most alarms, using each series' length T, its alarm count N_T and $\gamma = 0.005$, next to the observed gap over the whole series (uncapped ADITI, so the theorem applies directly). Every one of the 32 series satisfies the bound, and the observed gaps (at most 0.026) are at least 5.6 times smaller than it, often far more: the bound is a worst case over adversarial sequences, and real streams are far from adversarial. The table also lists K_T, the number of record residuals that enter Corollary 1. Records occur at an average rate of 0.0032 per step, about three times the i.i.d. rate of $1/1001$, because operational residuals are heavy-tailed and drifting; the cap's added term remains small.

Appendix G: Where the Detector Fires

Fig. 23 places every alarm on the 15 test series that had any, against NAB's labelled windows. Alarms cluster on the two failure series of Section VII-E and on the other disk-write counter, which together account for 89 of the 156 alarms. On the tweet and traffic series, alarms are sparse and fall both inside and outside windows, consistent with the 25% in-window share reported in Section VII-D.6.

Table XIX
PER-SERIES TEST RESULTS (NAIVE FORECASTER, $1-\alpha=0.9$)

		Local coverage error ↓					Interval score / Rolling ↓				ADITI
Series	Cat.	Roll.	ACI	DtACI	QT	ADITI	ACI	DtACI	QT	ADITI+cap	in win.
ec2_cpu_utilization_fe7f93	AWS	0.032	0.021	**0.010**	0.013	0.013	0.92	0.78	0.90	0.90	0.891
ec2_cpu_utilization_ac20cd	AWS	0.022	0.019	0.010	**0.007**	0.008	1.00	1.01	0.98	0.98	0.903
rds_cpu_utilization_e47b3b	AWS	0.025	0.015	0.017	**0.008**	0.011	1.00	1.00	1.02	1.02	0.893
elb_request_count_8c0756	AWS	0.016	0.013	0.013	0.009	**0.009**	1.00	0.99	0.99	1.00	0.887
grok_asg_anomaly	AWS	0.021	0.015	0.014	**0.013**	0.014	0.99	1.03	1.00	1.03	0.885
ec2_cpu_utilization_c6585a	AWS	0.058	0.008	0.013	**0.001**	0.020	1.00	1.00	1.11	1.04	–
ec2_disk_write_bytes_c0d644	AWS	0.045	0.044	**0.032**	0.043	0.040	0.96	0.86	0.94	1.02	0.953
iio_NetworkIn	AWS	0.034	0.022	0.010	**0.008**	0.012	0.95	0.89	0.94	0.93	0.935
ec2_disk_write_bytes_1ef3de	AWS	0.055	0.052	**0.043**	0.047	0.049	0.96	0.87	0.94	0.89	0.953
ec2_cpu_utilization_53ea38	AWS	0.012	0.009	0.011	**0.006**	0.009	1.00	1.00	1.01	1.01	0.897
rds_cpu_utilization_cc0c53	AWS	0.022	0.012	0.013	**0.009**	0.010	1.01	0.98	1.00	1.01	0.907
exchange-3_cpc_results	Ad	0.019	0.014	0.014	**0.007**	0.010	0.99	0.98	1.00	0.99	0.897
exchange-4_cpm_results	Ad	0.030	0.019	0.012	**0.008**	0.018	1.00	1.01	1.02	1.00	0.887
exchange-2_cpm_results	Ad	0.012	0.009	0.009	**0.007**	0.008	1.00	0.99	1.01	1.02	0.900
exchange-3_cpm_results	Ad	0.029	0.021	0.015	**0.007**	0.010	0.99	1.04	0.98	0.98	0.895
machine_temperature_system_failure	KC	0.020	0.014	0.015	**0.009**	0.009	1.00	0.99	1.00	1.00	0.896
ambient_temperature_system_failure	KC	0.015	0.011	0.013	**0.007**	0.009	1.00	1.00	1.01	1.01	0.907
cpu_utilization_asg_misconfiguration	KC	0.024	0.014	0.019	0.015	**0.009**	1.00	1.00	1.00	1.02	0.899
ec2_request_latency_system_failure	KC	0.016	0.011	0.014	**0.007**	0.010	1.00	0.99	1.00	1.00	0.881
rogue_agent_key_updown	KC	0.059	0.077	**0.051**	0.077	0.064	0.97	1.73	0.95	1.08	0.951
occupancy_t4013	Tr	0.022	0.015	0.012	**0.009**	0.010	1.00	0.99	0.99	1.01	0.867
speed_t4013	Tr	0.027	0.020	**0.009**	0.011	0.010	1.01	0.98	0.99	1.00	0.919
TravelTime_387	Tr	0.039	0.026	0.014	**0.008**	0.010	0.98	1.14	0.97	0.96	0.898
speed_7578	Tr	0.012	0.011	0.012	0.010	**0.008**	1.00	0.95	1.00	0.98	0.821
speed_6005	Tr	0.018	0.012	0.012	0.008	**0.007**	1.00	1.00	1.01	1.01	0.908
Twitter_volume_KO	Tw	0.024	0.020	0.009	**0.009**	0.012	0.97	1.96	0.98	0.98	0.904
Twitter_volume_IBM	Tw	0.027	0.023	**0.009**	0.011	0.013	0.98	0.99	0.95	0.96	0.904
Twitter_volume_CRM	Tw	0.033	0.025	**0.009**	0.014	0.013	0.99	1.48	0.94	0.95	0.901
Twitter_volume_CVS	Tw	0.043	0.025	**0.013**	0.014	0.020	1.01	1.14	0.96	1.04	0.888
Twitter_volume_GOOG	Tw	0.025	0.020	0.012	**0.010**	0.010	0.97	0.97	0.98	0.99	0.896
Twitter_volume_AAPL	Tw	0.031	0.021	0.011	**0.008**	0.013	0.92	1.29	0.95	0.96	0.900
Twitter_volume_AMZN	Tw	0.019	0.015	0.012	**0.008**	0.010	1.00	1.00	0.99	0.99	0.900

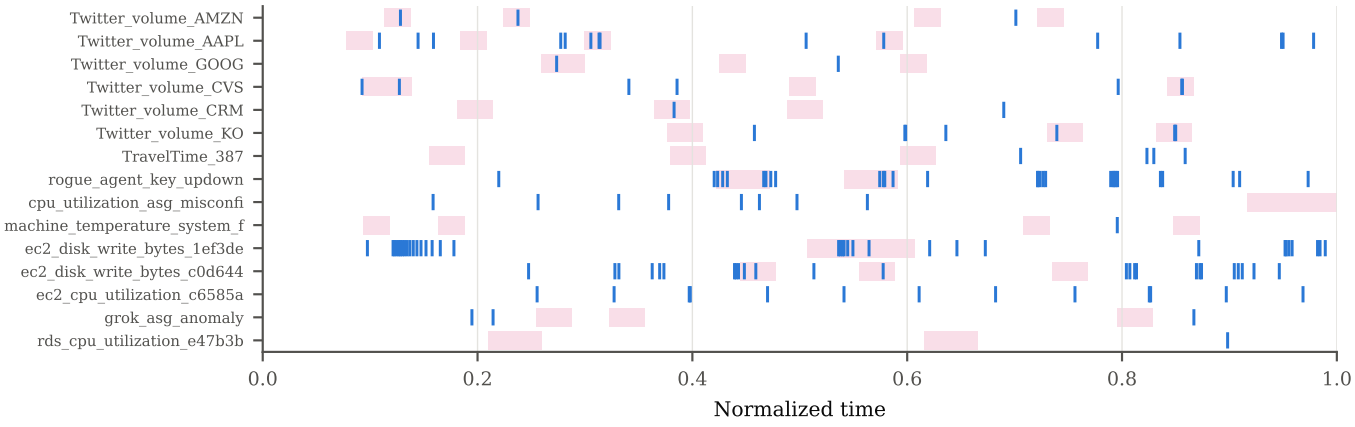


Figure 23. Drift alarms (ticks) and labelled anomaly windows (shaded) on the 15 test series with at least one alarm (AR forecaster).

Table XX
COVERAGE BOUND VERSUS OBSERVED GAP (AR, UNCAPPED ADITI)

Series	T	N_T	K_T	Obs. gap	Thm. 1	Cor. 1
ec2_disk_write_bytes_1ef3de	4730	35	8	0.0156	1.537	1.598
rogue_agent_key_updown	5315	29	15	0.0012	1.140	1.225
ec2_disk_write_bytes_c0d644	4032	25	11	0.0075	1.303	1.374
Twitter_volume_AAPL	15902	15	27	0.0011	0.203	0.230
ec2_cpu_utilization_c6585a	4032	13	12	0.0062	0.701	0.743
cpu_utilization_asg_misconfiguration	18050	8	29	0.0009	0.101	0.115
Twitter_volume_KO	15851	8	24	0.0013	0.115	0.128
Twitter_volume_CVS	15853	7	30	0.0022	0.102	0.117
TravelTime_387	2500	4	13	0.0112	0.404	0.430
grok_asg_anomaly	4621	3	13	0.0089	0.175	0.186
Twitter_volume_AMZN	15831	3	25	0.0020	0.051	0.057
Twitter_volume_CRM	15902	2	28	0.0019	0.038	0.043
Twitter_volume_GOOG	15842	2	34	0.0018	0.038	0.045
rds_cpu_utilization_e47b3b	4032	1	10	0.0077	0.100	0.105
machine_temperature_system_failure	22695	1	47	0.0013	0.018	0.022
ec2_cpu_utilization_fe7f93	4032	0	17	0.0075	0.050	0.054

Table XXI
TEST RESULTS AT TARGET COVERAGE 0.95 (AR)

Method	Cov.	Local	Worst	Score gm	Score med.	Width	In win.	Recov.
Rolling split	0.949	0.0181	0.884	1.000	1.000	1.000	0.916	0.0
ACI	0.950	0.0131	0.904	1.018	0.986	0.921	0.934	0.3
DtACI	0.951	0.0111	0.913	1.094	0.990	0.805	0.938	0.0
Quantile tracking	0.949	0.0172	0.884	0.982	0.978	1.002	0.916	0.2
ADITI	0.950	0.0083	0.922	2.129	0.941	0.793	0.947	0.0

Table XXII
TEST RESULTS AT TARGET COVERAGE 0.80 (AR)

Method	Cov.	Local	Worst	Score gm	Score med.	Width	In win.	Recov.
Rolling split	0.803	0.0411	0.654	1.000	1.000	1.000	0.746	34.3
ACI	0.800	0.0317	0.684	1.003	0.994	0.946	0.759	31.6
DtACI	0.804	0.0185	0.745	1.180	0.995	0.907	0.781	52.5
Quantile tracking	0.799	0.0141	0.734	0.987	0.989	0.981	0.776	72.9
ADITI	0.797	0.0186	0.732	2.019	0.975	0.923	0.813	80.3

APPENDIX H: FULL RESULTS AT 95% AND 80% TARGETS

Tables XXI and XXII give every metric behind Table XI (AR forecaster, ADITI as selected, without the cap; its geometric-mean score is inflated by the two failure series as in Table V, so the median is the better efficiency summary). At 95% ADITI has the narrowest bands (width 0.793 of rolling) and the best worst-case local coverage (0.922). At 80% its recovery after windows is the slowest (80.3 steps); we attribute this to its short memory fitting the incident regime and then being slow to let it go, but we have not verified that mechanism.

APPENDIX I: IMPLEMENTATION DETAILS

All methods are implemented in about 300 lines of Python with NumPy and share the residual stream $s_t = |y_t - \hat{y}_t|$, a 300-step warm-up and the same metric code. *Finite-sample quantile.* Rolling, ACI and DtACI use the $\lceil (n+1)(1-\alpha_t) \rceil$-th smallest of the last $n \le W$ scores, returning $+\infty$ when that index exceeds n and 0 when $\alpha_t \ge 1$. *DtACI.* Eight ACI experts with $\gamma \in \{0.001, 0.002, \ldots, 0.128\}$ share one score window. At each step the issued level is the weight-averaged expert level. After observing s_t, each expert's loss is the pinball loss of its level against β_t, the largest miscoverage level whose interval still covers s_t; weights are updated by exponential weights with the learning rate of [6] for interval length $I = 500$ and mixed with the uniform distribution at rate $1/(2I)$; each expert then takes its own ACI step. *Quantile tracking.* The threshold starts at the split-conformal quantile of the warm-up scores and moves by $\eta B_t(\mathrm{err}_t - \alpha)$, where B_t is the largest of the last 500 scores; the issued threshold is clipped at zero. We use the P-control part only, without the integral or scorecasting terms of [7]. *ADITI.* The memory holds the last $W = 1000$ normalized scores, initialized with the warm-up scores divided by their median. The scale starts at the warm-up median. The CUSUM standardizes $\log(r_t + 10^{-6})$ with exponentially weighted moments at rate 0.005, initialized from the warm-up. Each score is normalized by the scale *before* that score updates it, so the interval at time t uses only information up to $t-1$. *AR forecaster.* Recursive least squares on 12 lags plus intercept, with forgetting factor 0.999 and initial inverse covariance $100I$, applied to the series standardized by slow running moments (rate 0.001); weights are reset if they become non-finite or a standardized forecast exceeds 50 in magnitude. *Parallelism and seeds.* The NAB split uses seed 0; the controlled experiments use seeds 1000–1039. Runs are deterministic given the seed.